

CI와 GitHub Actions

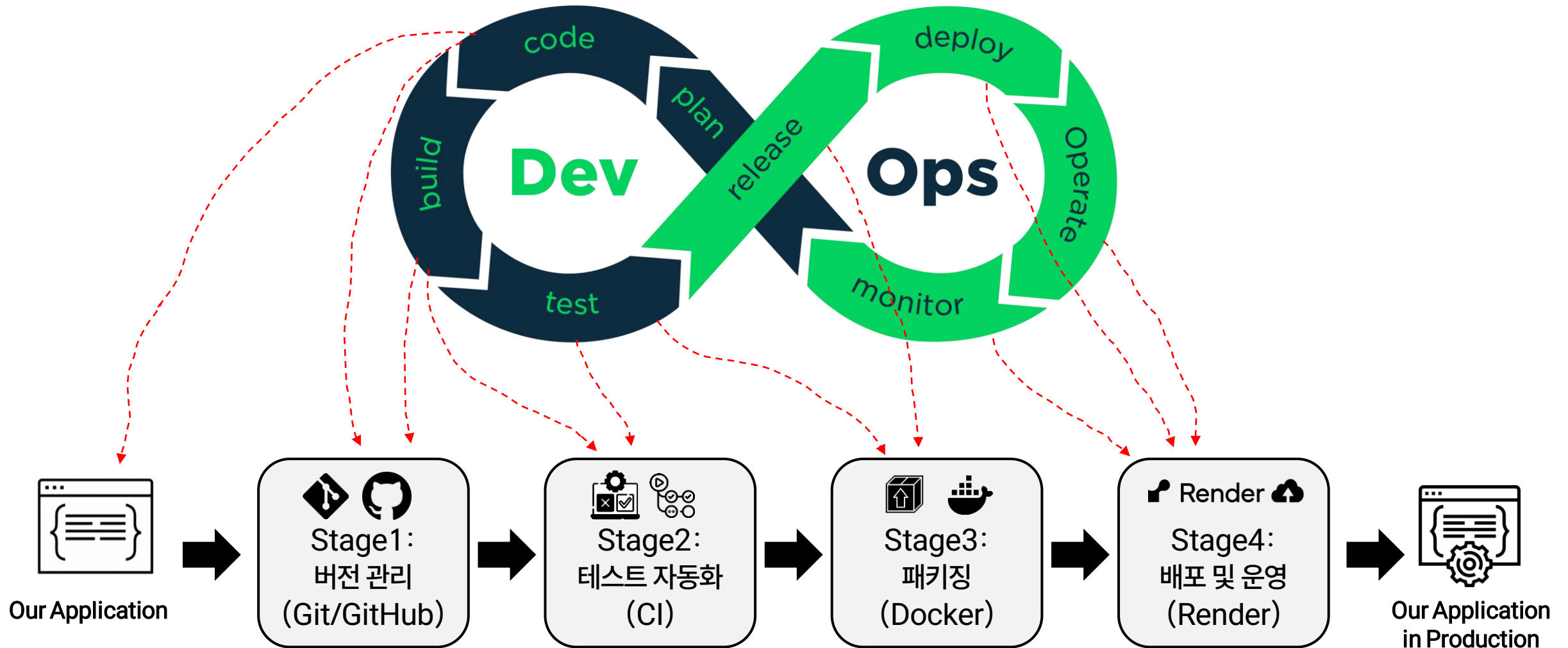
김미수

전남대학교 AI융합대학 인공지능학부

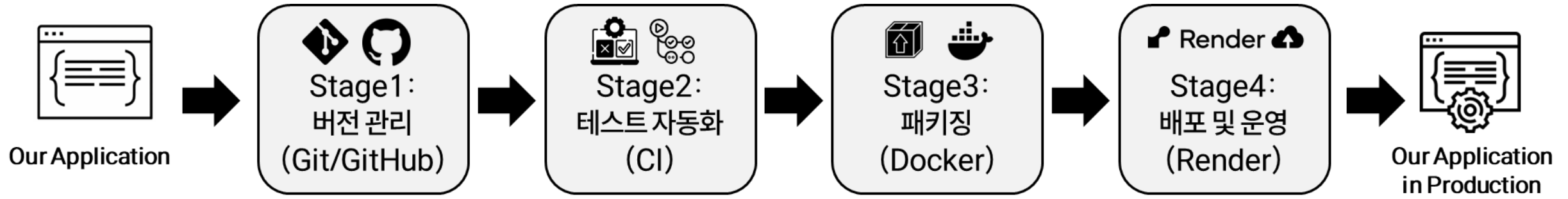
misoo.kim@jnu.ac.kr

<https://sites.google.com/view/semi-jnu>

수업에서 진행할 파이프라인



DevOps 파이프라인



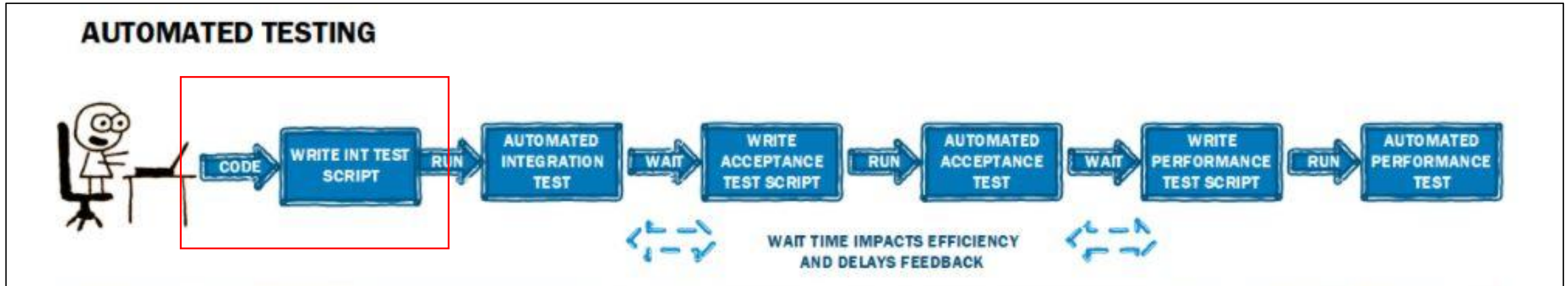
• 1단계: Git/GitHub & CI/테스트

- Git/GitHub: 로컬 코드를 팀과 공유하고, 변경 사항을 기록하고 관리
 - Git: 변경 기록, GitHub: 변경 사항의 원격 저장소
- CI: 코드 통합 시 자동으로 테스트해 빌드가 깨지는 것 방지
- 사용 Tool: GitHub Actions



테스트 자동화

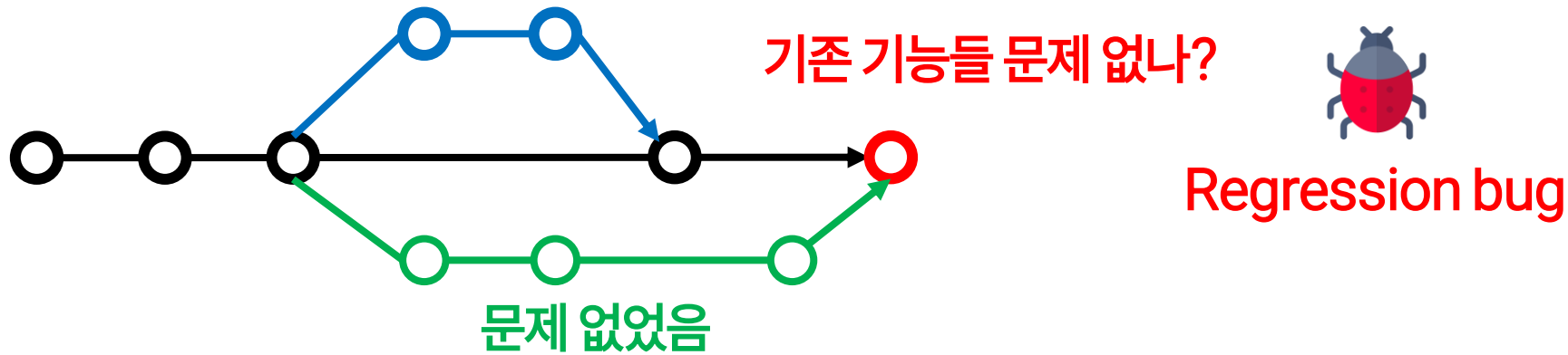
- 도구나 코드 (테스트 스크립트)를 사용해 테스트를 자동화 하는 것
 - 단계: 테스트 코드 작성 → 자동으로 실행 및 관리



- 개발자 입장에서 충분한 테스트 코드를 만들어서 자동화해 놓으면 → 수동 작업 축소!

Continuous Integration (CI), 지속적 통합

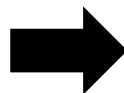
- 코드 변경이 발생할 때마다 자동으로 빌드 및 테스트를 수행하는 것
 - 기존: 개발자마다 로컬에서 테스트 → 다른 사람들의 코드와 통합될 때의 이슈 보장 안됨



Continuous Integration (CI), 지속적 통합

- Regression bug: 새로운 수정 때문에 기존에 정상 동작하던 기능이 다시 고장나는 버그

```
1 # calc.py
2 def calculate_price(price, is_vip):
3     if is_vip:
4         return price * 0.8 # VIP는 20% 할인
5     return price
6
7 # calc_test.py
8 assert calculate_price(100, True), 80.0 # 80.0
9 assert calculate_price(100, False), 100.0 # 100.0
10
```



```
1 # calc.py
2 def calculate_price(price, is_vip, has_coupon=False):
3     if has_coupon:
4         return price * 0.9 # 쿠폰 10% 할인
5     elif is_vip:
6         return price * 0.9 # 버그! 원래 VIP는 20% 할인이어야 했음
7     return price
8
9 # calc_test.py
10 assert calculate_price(100, True), 80.0 # FAIL: 90.0
11 assert calculate_price(100, False), 100.0 # 100.0
12
```

```
1 # auth.py
2 def login(username, password):
3     if username == "admin" and password == "1234":
4         return "success"
5     return "fail"
6
7 # auth_test.py
8 assert login("admin", "1234"), True
9
```

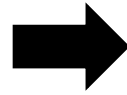


```
1 # auth.py
2 def login(username, password):
3     if len(password) < 6:
4         return "fail"
5     if username == "admin" and password == "1234":
6         return "success"
7     return "fail"
8
9 # auth_test.py
10 assert login("admin", "1234"), True # FAIL
```

Continuous Integration (CI), 지속적 통합

- Regression bug: 새로운 수정 때문에 기존에 정상 동작하던 기능이 다시 고장나는 버그

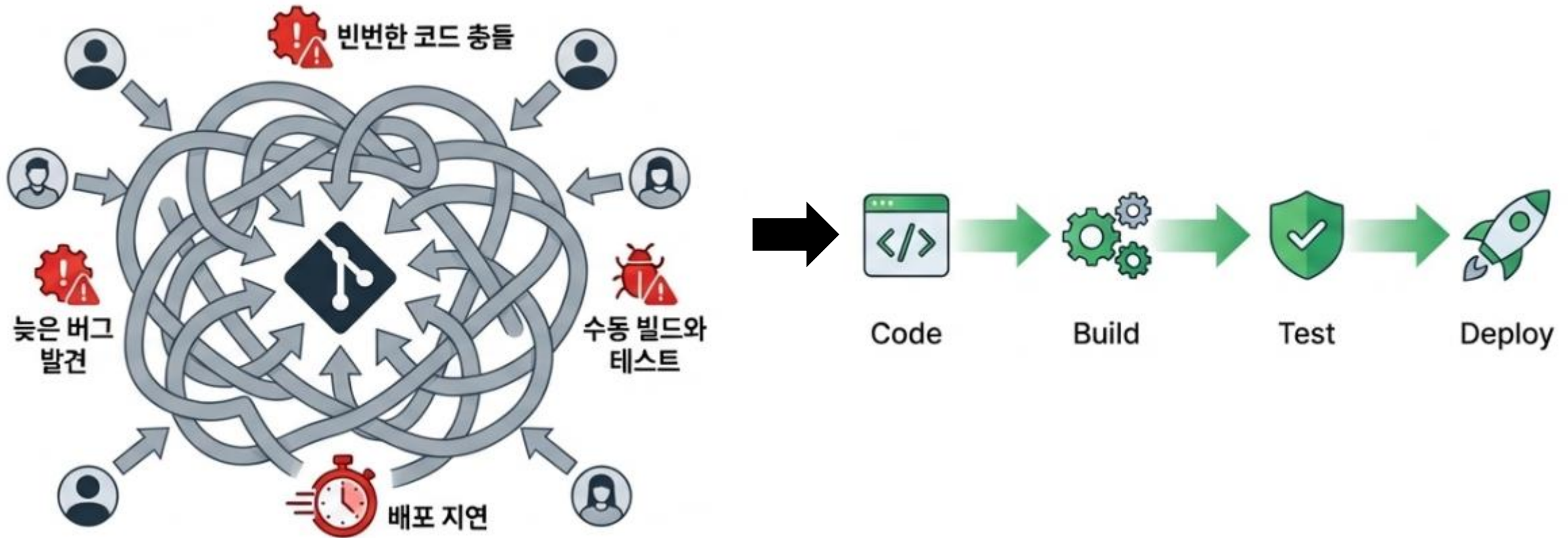
```
1 # login.py
2 def login(username, password):
3     if username == "admin" and password == "1234":
4         return {"status": "success", "role": "admin"}
5     return {"status": "fail"}
6
7 # dashboard.py
8 def access_dashboard(user_info):
9     if user_info["status"] != "success":
10         return "Access Denied"
11
12     if user_info["role"] == "admin":
13         return "Admin Dashboard"
14     return "User Dashboard"
15
16 # main.py
17 user = login("admin", "1234")
18 print(access_dashboard(user))
19 # Admin Dashboard!
20
```



```
1 # login.py
2 def login(username, password):
3     # 새로운 정책
4     if len(password) < 6:
5         return {"status": "fail"}
6
7     if username == "admin" and password == "1234":
8         return {"status": "success", "role": "admin"}
9
10    return {"status": "fail"}
11
12 # dashboard.py
13 def access_dashboard(user_info):
14     if user_info["status"] != "success":
15         return "Access Denied"
16
17     if user_info["role"] == "admin":
18         return "Admin Dashboard"
19     return "User Dashboard"
20
21 # main.py
22 user = login("admin", "1234")
23 print(access_dashboard(user))
24 # Access Denied!
```

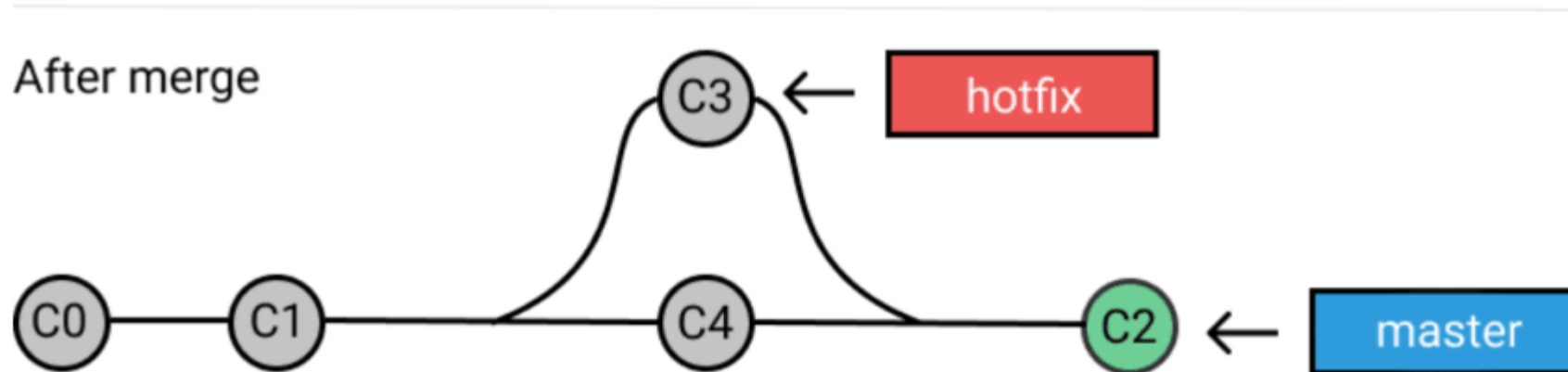
Continuous Integration (CI), 지속적 통합

- 코드 변경이 발생할 때마다 자동으로 빌드 및 테스트를 수행하는 것
 - 기존: 개발자마다 로컬에서 테스트 → 다른 사람들의 코드와 통합될 때의 이슈 보장 안됨



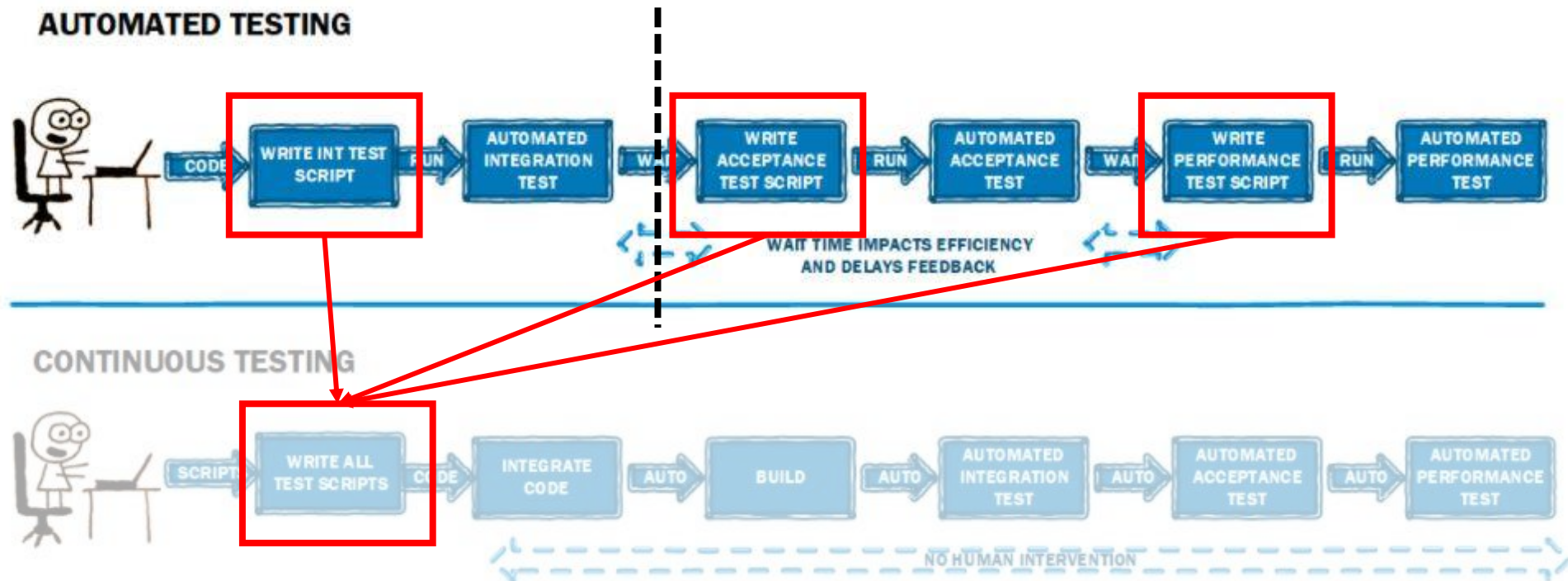
Continuous Integration (CI), 지속적 통합

- 새로운 코드 변경 사항이 지속적으로 빌드/테스트 되어 **공유 저장소**에 통합
- 필수 환경 1. 다수의 개발자가 공통의 SW 형상 관리 툴을 공유하여 사용
 - SW 형상 관리 (SCM, SW Configuration Management) (= 버전 관리)
 - 변경 사항을 체계적으로 추적, 통제하여 소프트웨어 형상(구성)을 관리하는 것
 - 대표적인 도구: Git, SVN 등 + GitHub (저장소)



Continuous Integration (CI), 지속적 통합

- 새로운 코드 변경 사항이 지속적으로 **빌드/테스트** 되어 공유 저장소에 통합
- 필수 환경 2. 지속적 테스트 (Continuous test) 환경
 - 당연히 테스트 자동화 환경이 구축되어야 함



Continuous Integration (CI), 지속적 통합

- CI: 테스트를 자동으로 실행시키면서 “올바른 통합을 보장”
 - 테스트 자동화: 테스트를 자동으로 실행, CI의 핵심 요소
- 핵심 개념
 - Trigger: CI를 수행시킬 입력
 - Pipeline: Build → test → report
 - Feedback: 성공 or 실패
 - Integration 문제 해결: merge conflict, dependency 문제, regression bug 등

Continuous Integration (CI), 지속적 통합

- CI: 테스트를 자동으로 실행시키면서 “올바른 통합을 보장”
 - 여러 개발자가 작업한 코드를 정기적으로 중앙 저장소에 통합하는 개발방식
 - 코드 통합 문제를 조기에 발견하고 해결해,
소프트웨어 품질을 향상시키고 개발 시간을 단축
 - “모든 프로세스는 자동화가 되어야 함”

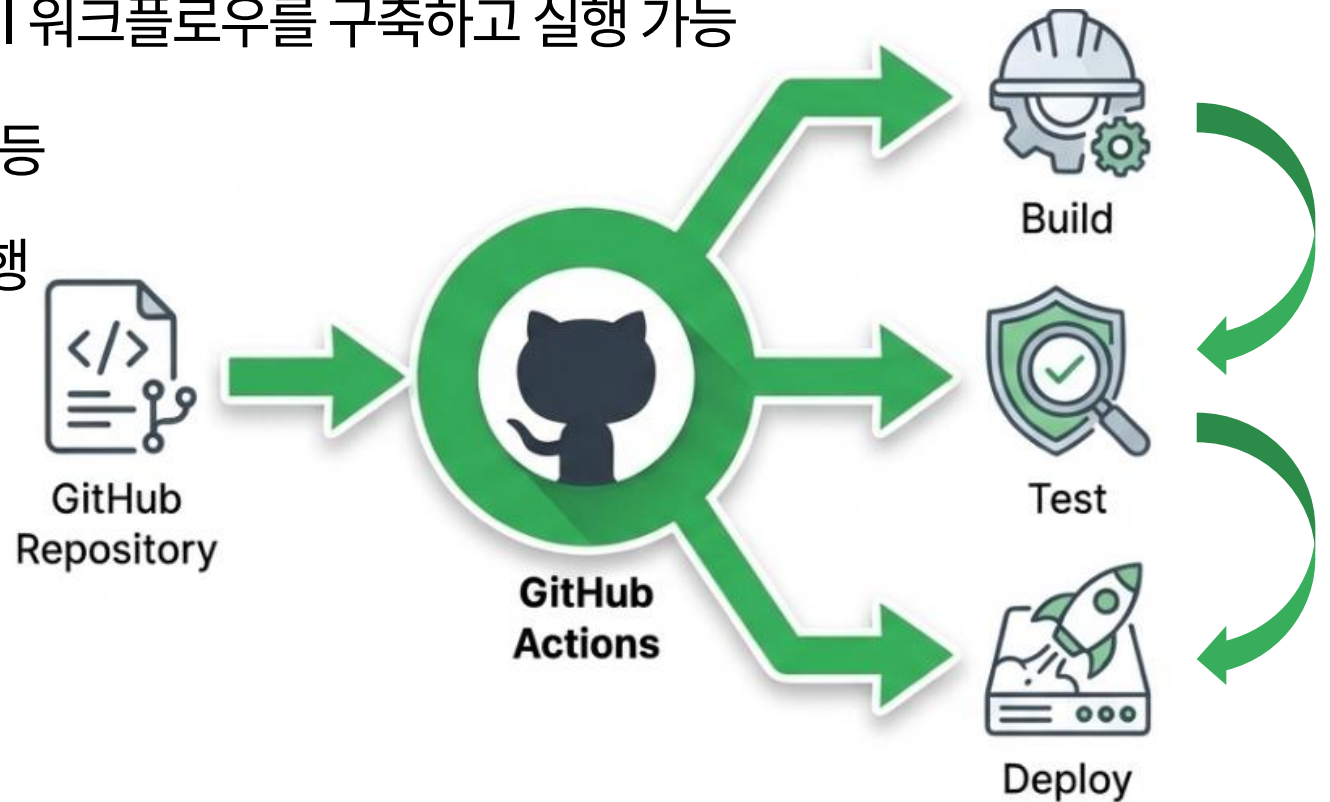
▶ 우리 수업에서는 GitHub Actions!



GitHub Actions

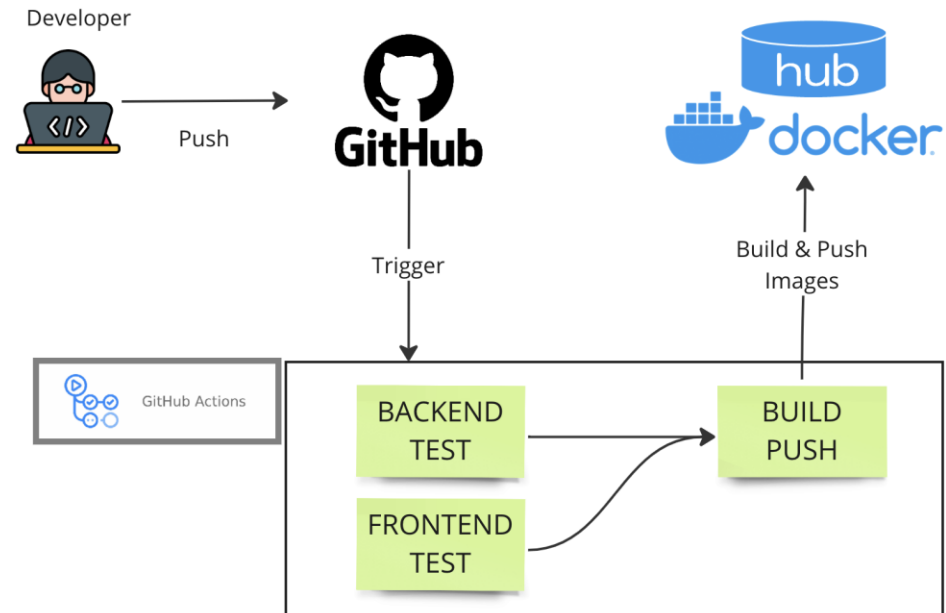
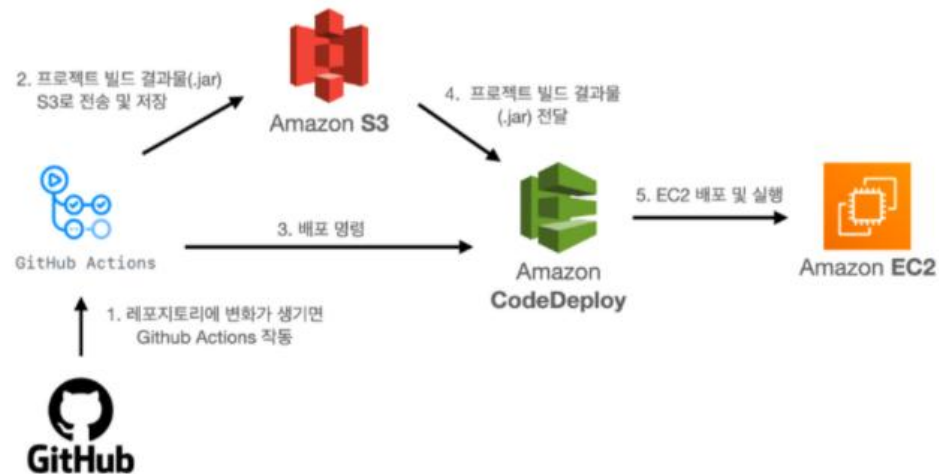
- CI를 자동화하기 위한 도구

- GitHub 네이티브: GitHub 저장소에서 CI 워크플로우를 구축하고 실행 가능
- 워크플로우 자동화: push, pull request 등 github 이벤트에 따른 작업을 자동으로 실행
- YAML 기반: 사람이 읽고 쓰기 쉬운
YAML 파일로 워크플로우 정의
- 풍부한 생태계: 커뮤니티 내 수많은 Actions를 재사용 가능



GitHub Actions

- GitHub에 통합된 자동화 도구로 CI/CD를 자동화할 수 있음
 - 설명서: <https://docs.github.com/ko/actions>

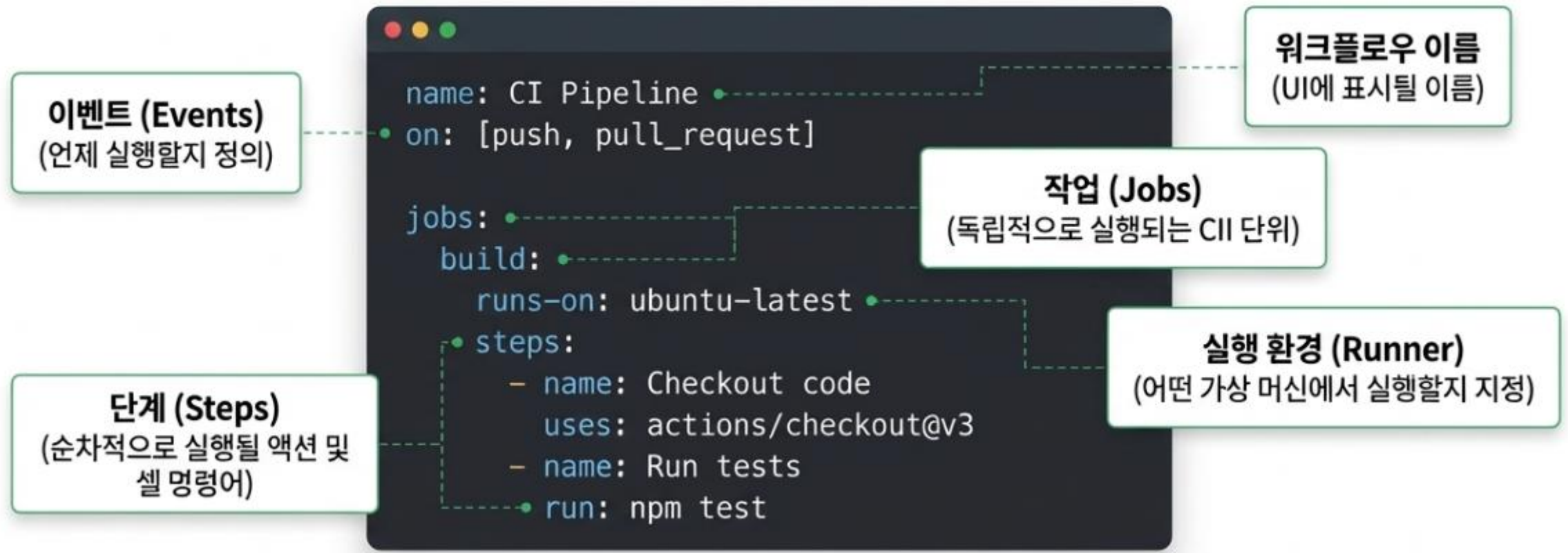


GitHub Actions

- GitHub에서 “이벤트”가 발생하면 그에 따른 워크플로우를 실행
- 구성요소
 - 이벤트(Event): 무엇을 trigger 할 것인가 (e.g., push, pull_request 등)
 - 워크플로우(Workflow): 이벤트가 발생했을 때 수행할 자동화 시나리오
 - 잡 (Job): 워크플로우 안에서 실행될 작업 묶음
 - 스텝 (Step): 각 잡 안의 실제 명령어 실행 단계
 - 액션 (Action): 미리 만들어진 자동화 기능
 - <https://github.com/marketplace?type=actions>

GitHub Actions

- YAML 파일 해부: CI 파이프라인 설계도



GitHub Actions

• CI 자동화 워크플로우 예시

```
# .github/workflows/ci.yml

name: Python CI

on:
  push:
    branches: [ "main" ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-python@v3
        with:
          python-version: "3.9"
      - run: pip install -r requirements.txt
      - run: pytest
```

1.

1. 리포지토리 코드를 가져옵니다.
(Checkout)

2.

2. 특정 버전의 Python을 설치합니다.
(Setup Python)

3.

3. 필요한 라이브러리를 설치합니다.
(Install Dependencies)

4.

4. 자동화된 테스트를 실행합니다.
(Run Pytest)

이 설계도는 'main' 브랜치에 코드가 푸시될 때마다 자동으로
실행되며, 코드의 무결성을 완벽하게 검증합니다.

GitHub Actions

- CI 자동화 워크플로우 실행 결과

The screenshot displays a GitHub Actions workflow run for the repository 'github.acs / Workflow-runs'. The workflow is named 'Add feature X' and was triggered by a push to the 'a8b1c2d' branch 3 hours ago. The run status is 'Success', indicated by a green checkmark. A list of steps is shown, all of which completed successfully:

- ✓ Build
- ✓ Checkout code
- ✓ Set up Python
- ✓ Install dependencies
- ✓ Run tests

Three green callout boxes provide additional context:

- 성공: 모든 빌드와 테스트 통과!** (Success: All build and tests passed!) - Points to the 'Success' status.
- 어떤 코드 변경에 의해 실행되었는지 명확하게 확인 가능** (It is possible to clearly confirm which code change triggered the execution) - Points to the commit hash 'a8b1c2d'.
- 문제 발생 시, 어떤 단계에서 실패했는지 즉시 파악 가능** (When a problem occurs, it is possible to immediately identify in which step it failed) - Points to the list of steps.

GitHub Actions

 `.github/workflows/python-test.yml` 워크플로우: 자동화 전체 시나리오를 정의한 yaml 파일 ▶ GitHub 내에서 이 파일들 자동으로 실행

```
name: Python Test Workflow
```

`on: [push]` 이벤트: 무슨 일이 생겼을 때 자동화 할 것인가? = push 이벤트 발생시

`jobs:` 잡: 하나의 독립적인 실행 환경. 작업의 묶음

```
test: (job의 이름)
```

```
runs-on: ubuntu-latest (이 job이 실행될 OS 환경)
```

`steps:` 스텝: 이 잡 안에서 실행되는 개별 작업들

```
- uses: actions/checkout@v3 (미리 정의된 액션들 중 GitHub 코드 체크아웃 액션을 사용)
```

```
- uses: actions/setup-python@v4
  with: (액션들 중 Python 설치 액션 사용)
    python-version: '3.10'
```

```
- name: Install dependencies (미리 정의된 액션은 아니고, 자체적으로 cmd에서 실행할 명령어)
  run: pip install -r requirements.txt
```

```
- name: Run tests (미리 정의된 액션은 아니고, 자체적으로 cmd에서 실행할 명령어)
  run: pytest
```

GitHub 내에서 Push 이벤트 발생 시,
해당 버전으로 checkout 하고,
자동으로 Python 환경을 세팅하고 (version 3.10)
requirement.txt에 정의된 필요한 의존성을 설치하고,
pytest로 테스트를 실행

How to integrate & automate in GitHub

- <https://github.com/keras-team/keras/issues/19156>

TorchDataLoaderAdapter is not compatible with torch.utils.data.IterableDataset (no `__len__()` dataset) New issue

#19156

Closed ixig opened this issue on Feb 7 · 3 comments · Fixed by #19176

ixig commented on Feb 7 · edited

The PyTorch DataLoader supports both map-style and iterable-style datasets -- the latter, IterableDataset, is used for data streams, and hence its length is unknown, and as such, has no `__len__()` method implemented. This is not unlike `tf.data.Dataset`, which also does not require that the dataset's length is known.

However, using an PyTorch IterableDataset fails with Keras -- at least it does when a PyTorch DataLoader created from an IterableDataset is passed as an argument to `validation_data` in `keras.Model.fit()`. There are (at least two) reasons:

1. In `JAXTrainer.fit()`, there is a conditional statement `if validation_data:` -- this causes Python to implicitly call `len()` on PyTorch's `DataLoader.__len__()`, which in turns call PyTorch's `IterableDataset.__len__()`
2. In `TorchDataLoaderAdapter.init()`, there is an explicit call: `self._size = len(data_loader)`

Also, an IterableDataset will return a batch of examples (instead of a single example) at a time, and as such, the associated DataLoader will be instantiated with `batch_size=None` (i.e. automatic batching disabled). As such, in `TorchDataLoaderAdapter.init()`, the following will fail:

```
self._batch_size = data_loader.batch_size
...
self._partial_batch_size = len(data_loader.dataset) % self._batch_size
```

Assignees

- hertschuh
- SuryanarayanaY

Labels

- backend:torch
- type:Bug

Projects

None yet

Milestone

No milestone

Development

Successfully merging a pull request may close this issue.

[Add support for IterableDataset to TorchDataLoaderAdapter · hertschuh/keras](#)

5 participants

github-actions bot assigned SuryanarayanaY on Feb 7

github-actions bot assigned SuryanarayanaY on Feb 7

sachinprasadhs added `keras-team-review-pending` `type:Bug` `backend:torch` labels on Feb 8

SamanehSaadat assigned hertschuh on Feb 9

SamanehSaadat removed the `keras-team-review-pending` label on Feb 9

hertschuh added a commit to hertschuh/keras that referenced this issue last month

[Add support for IterableDataset to TorchDataLoaderAdapter · 073afe6](#)

hertschuh mentioned this issue last month

[Add support for IterableDataset to TorchDataLoaderAdapter · #19176](#) Merged

GitHub action으로 부분적으로 CI 흉내내기

- 오래되어 버전은 좀 안 맞지만 참고할만한 자료
 - <https://github.com/cheese10yun/github-project-management>

목차

- [프로젝트 소개](#)
- [목차](#)
- [전체 플로우](#)
 - [Issue 발행](#)
 - [Issue란?](#)
 - [Issue Template](#)
 - [Issue Template 등록](#)
 - [Issue Template 사용법](#)
 - [Issue Template 파일](#)
 - [Issue 작업](#)
 - [등록된 issue 살펴 보기](#)
 - [Issue 연동](#)
 - [Issue 기반 Branch 생성](#)
 - [Pull Request\[Code Review\]](#)
 - [Jetbrains Pull Request](#)
 - [GitHub Pull Request](#)
 - [Pull Request 작성법](#)
 - [Code Review](#)
 - [IntelliJ 기반 Code Review](#)
 - [Code Review](#)
- [CI & Test Coverage](#)
 - [전체 플로우](#)
 - [Pull Request & Code Review](#)
 - [테스트 커버지리 표시](#)
- [Wiki](#)
 - [Wiki 등록](#)
 - [GitHub 작성](#)
 - [Local 작성](#)
- [ZenHub 사용법](#)
 - [기본 칸반보드](#)
 - [Milestone 적극 활용](#)
 - [Epic 적극 활용](#)
 - [Issue 연결](#)
 - [Filter 기능](#)
- [Github Action](#)
 - [Github Action 만들기](#)
 - [gradle.yml](#)
 - [Action Workflows](#)
 - [Badge](#)
 - [Schedule With Spring Batch](#)
 - [Schedule Github action 생성](#)
 - [Batch Code](#)

CI 자동화를 위한 연구들은 무엇이 있을까?

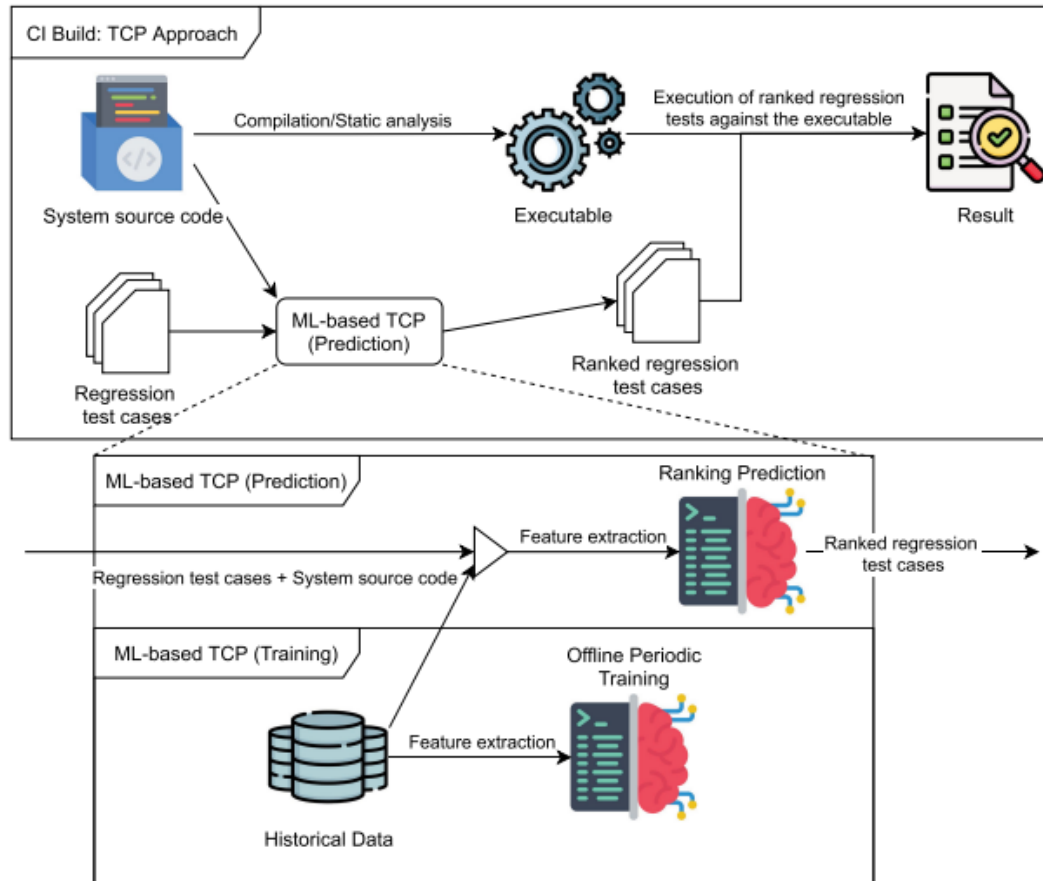
- Test case selection/prioritization
- 자동 코드 리뷰
- Flaky test 자동 탐지 및 수정 (APR)
- GitHub Actions 문제 해결 (APR)
- CI 빌드 실패 예측, 원인 분석 등등

Test Case Selection/prioritization 이란?

- 문제: CI 단계에서 모든 테스트 케이스를 돌리는 것은 많은 시간을 소모함 → 배포 느려짐
- 목표: 변경된 사항과 관련된 테스트만 선택(selection)하거나, 관련된 것부터 돌려서 (prioritization),
문제를 빨리 찾자!
- 방법론: 코드 커버리지 기반, dependency graph 기반 등등 → 머신러닝

Scalable and Accurate Test Case Prioritization in Continuous Integration Contexts, TSE' 23

- Test 관련 피처를 정의하고, 이를 머신러닝으로 학습
→ 점수가 높은 순서대로 TC를 먼저 돌리기!



Feature Group Pair		p-value	CL
COD_COV_COM ①	TES_CHN	0.00	1.00
	TES_PRO	0.00	0.98
	REC	0.00	0.95
	TES_COM	0.00	0.80
	F_COV	0.00	0.60
	COD_COV_CHN	0.00	0.59
	COD_COV_PRO	0.00	0.59
	DET_COV	0.00	0.59
COD_COV_PRO ②	TES_CHN	0.00	0.99
	TES_PRO	0.00	0.95
	REC	0.00	0.92
	TES_COM	0.00	0.71
	F_COV	0.00	0.51
	DET_COV	0.00	0.51
	COD_COV_CHN	< 0.01	0.50
DET_COV ③	TES_CHN	0.00	0.99
	TES_PRO	0.00	0.95
	REC	0.00	0.92
	TES_COM	0.00	0.71
	F_COV	0.00	0.51
	COD_COV_CHN	< 0.01	0.50
COD_COV_CHN ④	TES_CHN	0.00	0.99
	TES_PRO	0.00	0.95
	REC	0.00	0.92
	TES_COM	0.00	0.71
	F_COV	0.00	0.51
F_COV ⑤	TES_CHN	0.00	0.98
	TES_PRO	0.00	0.95
	REC	0.00	0.92
	TES_COM	0.00	0.70
TES_COM ⑥	TES_CHN	0.00	0.98
	TES_PRO	0.00	0.93
	REC	0.00	0.88
TES_PRO ⑦	TES_CHN	0.00	0.74
	REC	< 0.01	0.50
REC ⑧	TES_CHN ⑨	0.00	0.73

자동 코드 리뷰

- 배경: Pull request가 올라오면 개발자들간 코드 리뷰를 진행함 → Test 성공과 별개로 CI 문제를 미리 식별
- 문제: 수동으로 진행하기 어렵고 문제가 있을 수 있음 → 해결책: LLM Agent로 대체하자
- CodeAgent: Autonomous Communicative Agents for Code Review, EMNLP'24

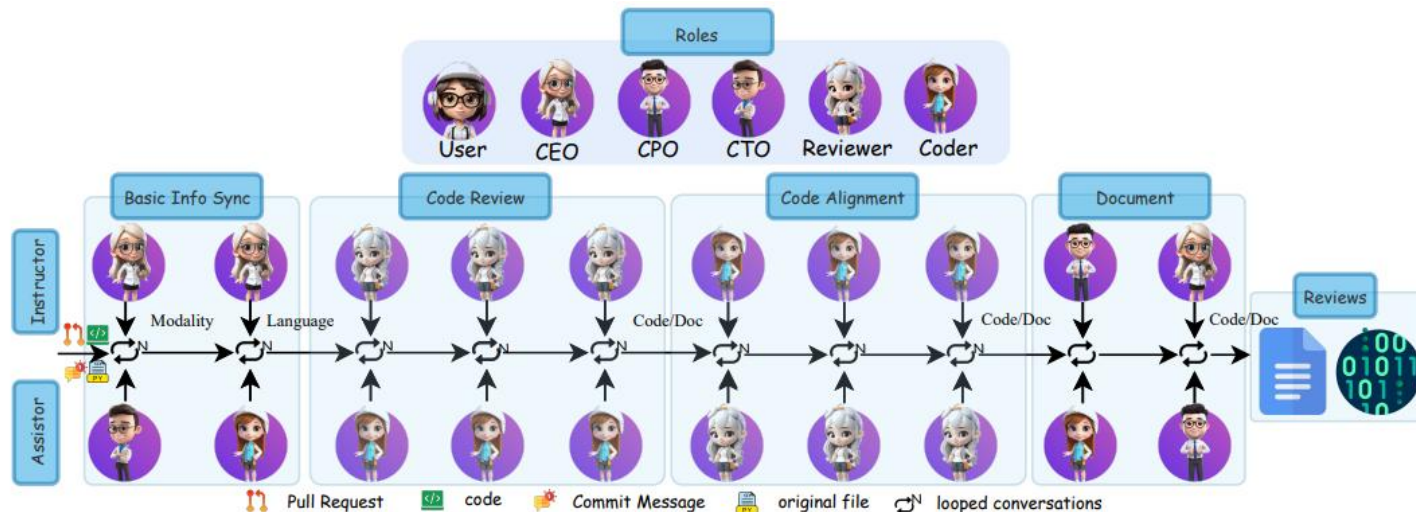
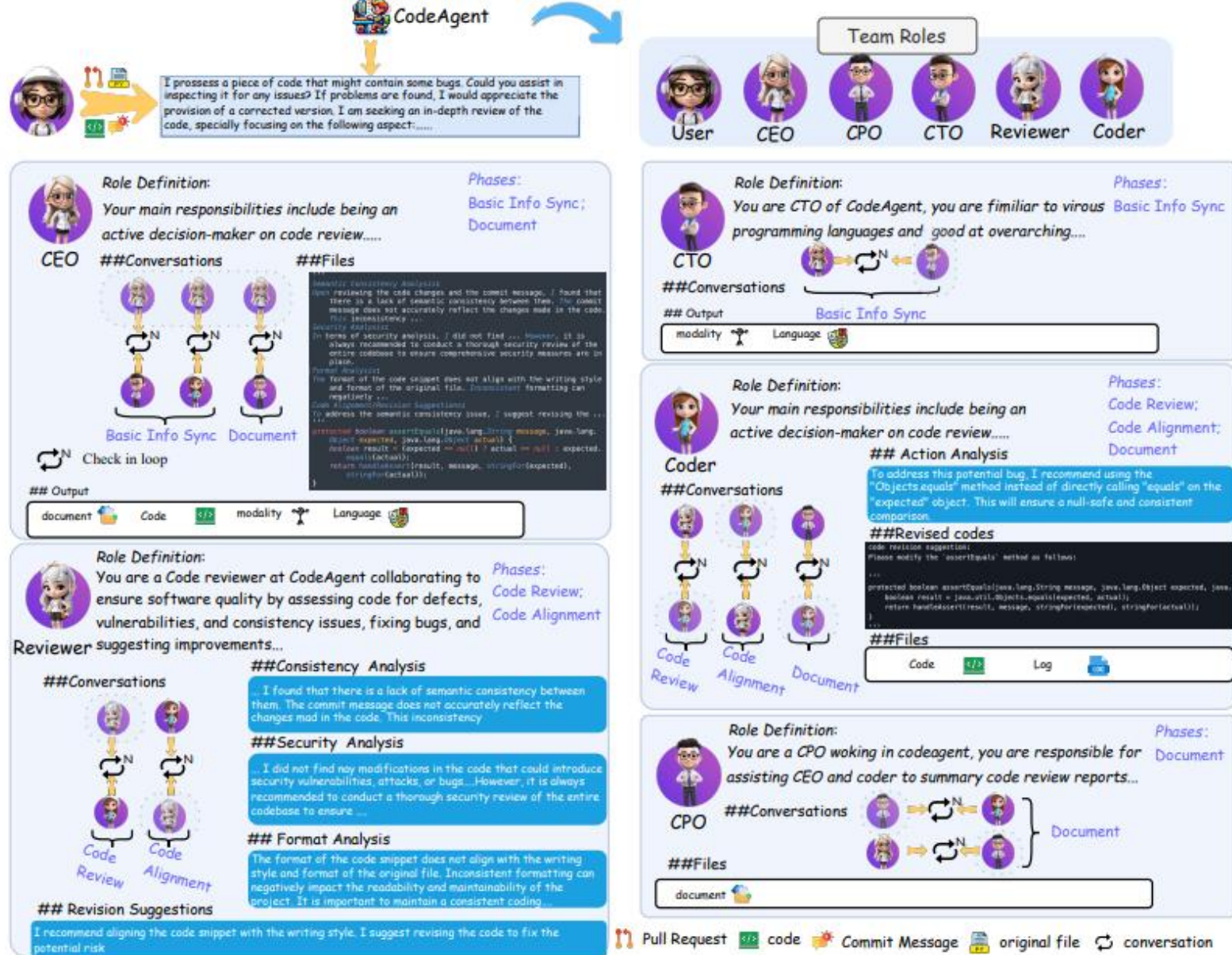


Figure 2: CodeAgent's pipeline/scenario of a full conversation during the code review process among different roles. "Basic Info Sync" demonstrates the basic information confirmation by the CEO, CTO, and Coder; "Code Review" shows the actual code review process; "Code Alignment" illustrates the potential code revision; and "Document" represents the summarizing and writing conclusion for all the stakeholders. All the conversations are being ensured by the Quality Assurance checker until they reach the maximum dialogue turns or meet all the requirements.

자동 코드

• CodeAgent



_P'24

Figure 1: A Schematic diagram of role data cards of simulated code review team and their conversations within CodeAgent. We have six characters in CodeAgent across four phases, including “Basic Info Sync”, “Code Review”, “Code Alignment”, and “Document”. Code review is a kind of collaboration work, where we design conversations between every two roles for every step to complete the task.

Automated Program Repair

- 주어진 입력(버그리포트, test 실행 결과 등)을 가지고 코드 상 문제를 찾고 고치는 분야

Global Styles: Unnecessary vertical scrollbar appears #65825

Closed #65875

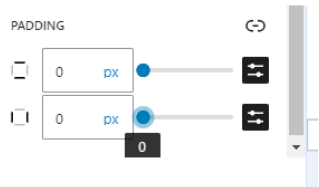
t-hamano opened last week

Description

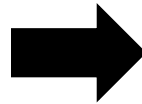
Note: This issue cannot be reproduced in WP6.7 Beta 1. It can be reproduced in the latest Gutenberg.

A vertical scrollbar may appear in global styles even when the content does not need to scroll vertically.

This issue seems to occur when there is a `RangeControl` at the end of the content. You can see that the tooltip that appears when operating the slider overflows:



The tooltip is there even though it is not visually visible, which is why the overflow appears to occur.



```
third_party/xla/xla/hlo/Utils/hlo_sharding_util.cc

@@ -582,12 +582,14 @@ std::optional<int64_t> SelectDominantDevice(
582     return count > 0 ? std::optional<int64_t>(device) :
      std::optional<int64_t>();
583 }
584
585 - HloSharding FindCommonSharding(absl::Span<const HloSharding> shardings) {
586     CHECK(!shardings.empty());
587     bool all_compatible = true;
588     HloSharding common_sharding = shardings[0];
589     for (int i = 1; i != shardings.size(); ++i) {
590         if (!MergeShardingIfCompatible(shardings[i],
591                                         common_sharding.NumTiles(),
592                                         &common_sharding)) {
593             all_compatible = false;
594             break;
595         }
596     }
597     return shardings[0];
598 }

+ HloSharding FindCommonSharding(absl::Span<const HloSharding> shardings,
+                                 std::optional<HloSharding> default_sharding)
+ {
587     CHECK(!shardings.empty());
588     bool all_compatible = true;
589     HloSharding common_sharding = shardings[0];
590     for (int i = 1; i != shardings.size(); ++i) {
591         if (common_sharding != shardings[i] &&
592             !MergeShardingIfCompatible(shardings[i],
593                                         common_sharding.NumTiles(),
594                                         &common_sharding)) {
595             all_compatible = false;
596             break;
597         }
598     }
599     // TODO(tongfei): instead of return the first sharding in case not all
600     // shardings are compatible, we should find a sharding that's compatible
601     // with the most number of shardings instead.
602     return default_sharding.has_value() ? default_sharding.value() :
603         shardings[0];
604 }
```

Flaky Test란? (불안정한 테스트)

- 같은 테스트를 여러 번 실행했을 때 결과가 달라지는 경우
 - 원인: 시간 의존성, 외부 시스템 의존성, 비결정성 (random) 등

```
import random

def get_number():
    return random.randint(1, 10)

def test_random():
    assert get_number() > 3
```

```
import time

def test_time():
    assert int(time.time()) % 2 == 0
```

```
1 @Test
2 public void testAutomaticStartStop() throws Exception {
3     final TestRunnable task = new TestRunnable(500);
4     e.execute(task);
5     Thread thread = e.thread;
6     assertThat(thread, is(not(nullValue())));
7     assertThat(thread.isAlive(), is(true));
8     Thread.sleep(1500);
9     assertThat(thread.isAlive(), is(false));
10    ...
11 }
```

(a) Original: Async Wait flaky test

```
1 async def test_get_image(hass, hass_ws_client, caplog):
2     """Test get image via WS command."""
3     await async_setup_component(
4         hass, "media_player", {"media_player": {"platform": "demo"}}
5     )
6     await hass.async_block_till_done()
7
8     client = await hass_ws_client(hass)
9
10    with patch(
11        "homeassistant.components.media_player.MediaPlayerEntity."
12        "async_get_media_image",
13        return_value=(b"image", "image/jpeg"),
14    ):
15        await client.send_json(
16            {
17                "id": 5,
18                "type": "media_player_thumbnail",
19                "entity_id": "media_player.bedroom",
20            }
21        )
22        msg = await client.receive_json()
23        assert msg["id"] == 5
24        assert msg["type"] == TYPE_RESULT
25        assert msg["success"]
26        assert msg["result"]["content_type"] == "image/jpeg"
27        assert msg["result"]["content"] == base64.b64encode(b"image").decode("utf-8")
28        assert "media_player_thumbnail is deprecated" in caplog.text
```

현재까지 등록된 task가 끝날 때 까지만 기다림

아래 작업이 원활이 안 끝날 가능성!

Fig. 5. A flaky test from the Home Assistant project [12]. This test was flaky until the addition of line 6 [13] (highlighted), which blocks the calling thread until all pending work to setup the components under test has been completed. Previously, the test might fail if this had not been completed before the assertions starting on line 23 were evaluated.

Flaky Test란? (불안정한 테스트)

- 같은 테스트를 여러 번 실행했을 때 결과가 달라지는 경우
- 연구 분야
 - Flaky test 탐지: 문제가 있는 test code 탐지
 - Flaky test 원인 분석: 어떤 종류의 flaky test인지 분류 (그래야 해결하니까)
 - Flaky test repair: 특정 flaky test를 자동으로 수정

Detection, Categorization and Repair of Flaky Tests Using Large Language Models, Ph.D Dissertation, Univ. of Ottawa, 2025

1. CodeBERT 모델로 테스트 코드가 flaky 한지 아닌지 분류
2. 어떤 종류의 Flaky test 인지 분류
3. 1, 2 정보와 함께 Flaky test code를 LLM에 넘겨서 자동 수정 요청

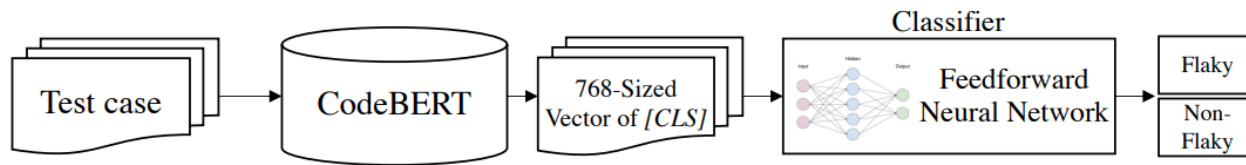


Figure 2.2: Fine-tuning CodeBERT for classifying test cases as flaky or not

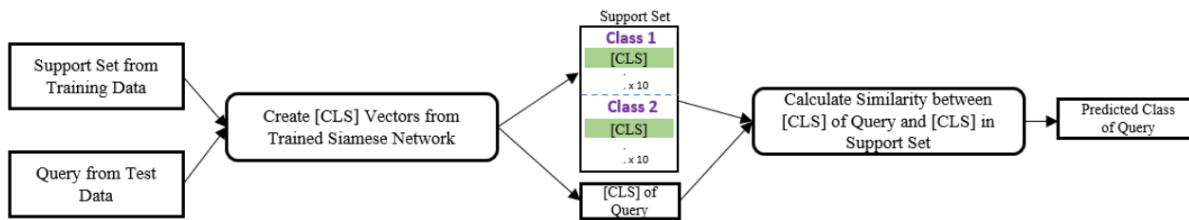


Figure 3.3: Predicting the class of query test case using few shot learning and a trained Siamese network.

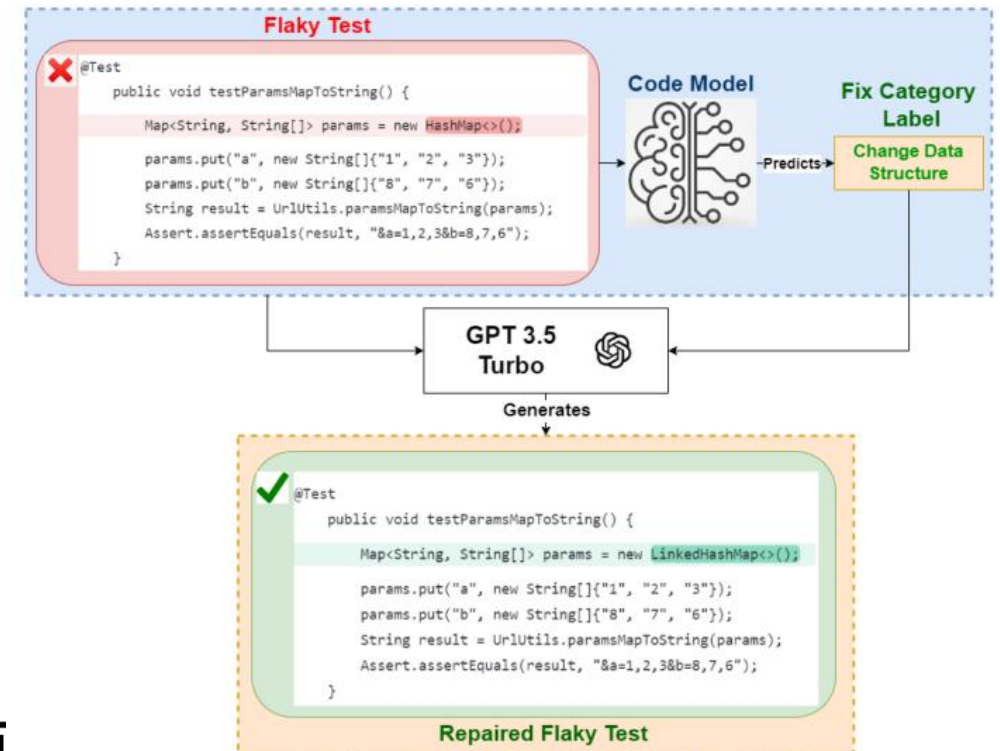


Figure 4.1: Predicting fix category label and generating repaired flaky tests using LLMs

GitHub Actions 관련 이슈

- Script 상 버그가 있을 수 있음

```
# .github/workflows/python-test.yml

name: Python Test Workflow

on: [push]

jobs:
  test:
    runs-on: ubuntu-latest

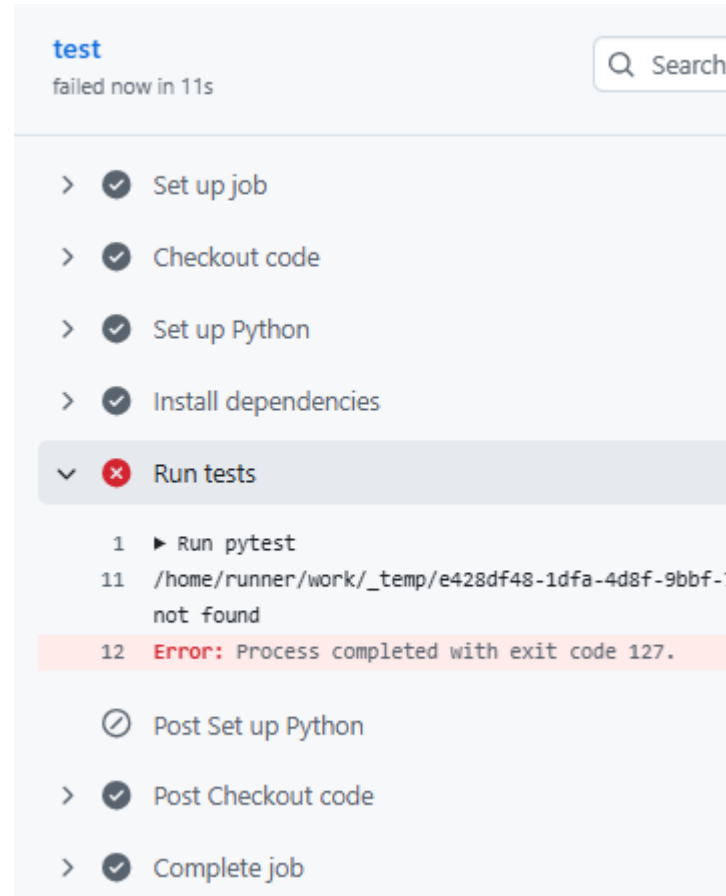
    steps:
      - uses: actions/checkout@v3

      - uses: actions/setup-python@v4
        with:
          python-version: '3.10'

      - name: Install dependencies
        run: pip install -r requirements.txt

      - name: Run tests
        run: pytest
```

- 작업 실패 시 문제 해결 필요



The screenshot shows a GitHub Actions workflow run named 'test' that failed. The status bar at the top indicates 'failed now in 11s'. A search bar is visible in the top right corner. The workflow steps are listed below:

- > ✓ Set up job
- > ✓ Checkout code
- > ✓ Set up Python
- > ✓ Install dependencies
- ▼ ✗ Run tests
 - 1 ▶ Run pytest
 - 11 /home/runner/work/_temp/e428df48-1dfa-4d8f-9bbf-...not found
 - 12 **Error:** Process completed with exit code 127.
- ⌚ Post Set up Python
- > ✓ Post Checkout code
- > ✓ Complete job

GitHub Actions 관련 이슈

- Script 상 버그가 있을 수 있음 → 연구를 위해서는 평가 데이터 셋이 필요
- GitBug-Actions: Building Reproducible Bug-Fix Benchmarks with GitHub Actions, ICSE-C'24

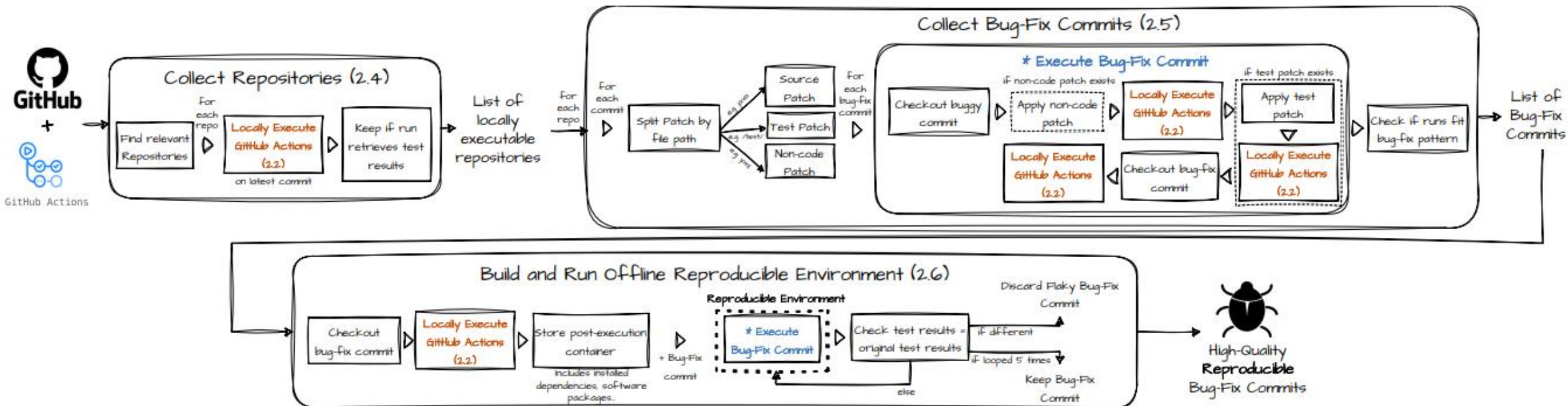


Figure 1: Overview of GitBUG-ACTIONS, a novel methodology to collect bug-fixes based on Github Actions.

GitHub Actions 관련 이슈

- Explaining GitHub Actions Failures with Large Language Models: Challenges, Insights, and Limitations, arxiv'25
 - GitHub Actions에서 에러 로그는 이해하기 어려움 → LLM으로 설명해서 개발자들을 돕자

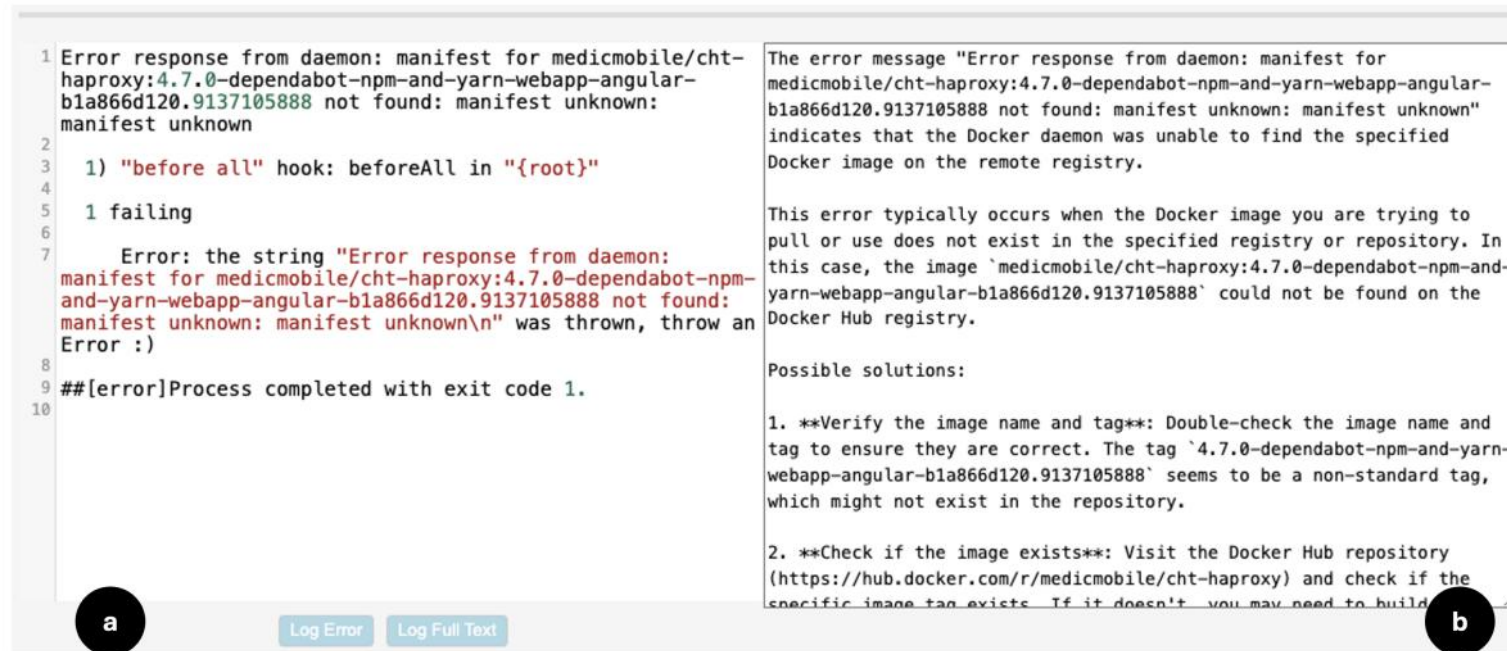


Fig. 2. Partial view of the LogExp tool's interface. The log is displayed on the left, allowing participants to choose between viewing a summary or the full log. On the right, the corresponding textual explanation generated by the LLM is presented. Below these sections, participants encountered statements and questions specific to each case.

실습 9. 스팸 체커 + CI

지난 시간까지..

- 공백 입력하면 안내 메시지가 뜨도록 수정!

스팸 판별기

스팸으로 의심되는 메시지를 붙여넣어주세요!

스팸 탐지

결과: undefined (score=undefined)

스팸 판별기

스팸으로 의심되는 메시지를 붙여넣어주세요!

스팸 탐지

메시지를 입력해 주세요!

- API 문서에 타입을 명확히 보이도록 수정

default

GET / Home

POST /classify Classify

Parameters

No parameters

Request body **required**

Example Value | Schema

```
{
  "text": "Win a FREE prize now, click!"
}
```

default

GET / Home

POST /classify Classify

Parameters

No parameters

Request body **required**

Example Value | Schema

Payload ^ Collapse all object

Additional properties allowed



default

GET / Home

POST /classify Classify

Parameters

No parameters

Request body **required**

Example Value | Schema

```
{
  "text": "string"
}
```

default

GET / Home

POST /classify Classify

Parameters

No parameters

Request body **required**

Example Value | Schema

ClassifyRequest ^ Collapse all object

```
text: string
```

이슈가 있다

`uvicorn app.main:app --reload`

- 근데 아직도 소스 코드에 문제는 있음!

- 이것저것 변경하다 보니 기존 기능 깨짐!

- 회귀 (regression) 문제

- 기존에 잘 동작하던 기능이

변경(추가/수정) 이후 의도치 않게 깨지는 현상

- 사람 눈으로 모두 다 직접 해볼 수 없음. 놓칠 수 있음
- 운영 시 문제가 커짐 (사용자 불만, 장애 대응, 신뢰 하락 등등)

▶ DevOps의 핵심: 변경 빨리 x → 변경을 안전하게

스팸 판별기

스팸으로 의심되는 메시지를 붙여넣어주세요

스팸 탐지

메시지를 입력해 주세요!

스팸 판별기

ddd

스팸 탐지

결과: undefined (score=undefined)

```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_
INFO: Will watch for changes in these directories: ['C:\\Users\\user\\OneDrive\\
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [3520] using StatReload
INFO: Started server process [2424]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:64831 - "GET / HTTP/1.1" 200 OK
INFO: 127.0.0.1:51491 - "POST /classify HTTP/1.1" 422 Unprocessable Content
□
```

테스트 자동화 필요성

- 테스트: 앞으로 코드가 바뀌어도 안 깨지게 지키는 안전장치
 - 테스트 = 문서, 계약(contract)
- 직접 하기 어렵다 ► 자동화 필요 = 테스트 코드 작성하고 실행시키자!

스팸 판별기

스팸으로 의심되는 메시지를 붙여넣어주세요:

스팸 탐지

메시지를 입력해 주세요!

스팸 판별기

ddd

스팸 탐지

결과: undefined (score=undefined)

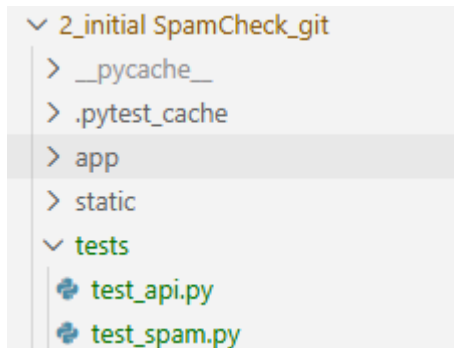
테스트 자동화 필요성

- 우리가 해볼 것: 단위 테스트와 API 테스트 (통합 테스트)

- 단위 테스트: 함수/모듈 하나를 떼어내서 로직이 맞는지 검증 → 회귀를 가장 싸게 잡는 1차 안전망
- API 테스트: 실제 엔드포인트 계약 (입출력)이 맞는지 검증 (요청 → 응답이 정상인가?)

- 테스트 코드 작성하고 실행시켜보기

- `pip install pytest httpx` (FastAPI test 용 라이브러리), requirements 갱신, gitignore 갱신
- Tests 폴더 만들고 test 코드 작성



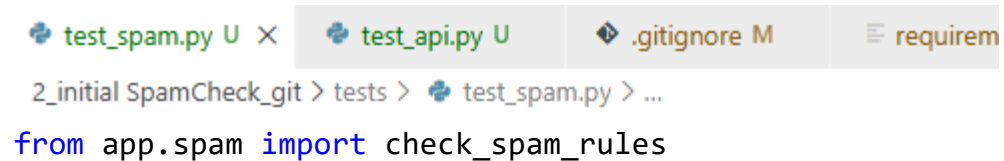
```
1 fastapi
2 uvicorn
3 python-multipart
4 pydantic
5 pytest
6 httpx
```

```
1 __pycache__/
2 *.pyc
3 .venv/
4 venv/
5 .env
6 .DS_Store
7 .idea/
8 .vscode/
9 .pytest_cache/
```

테스트 자동화 필요성

- 테스트 코드 작성하고 실행시켜보기

- pip install pytest httpx (FastAPI test 용 라이브러리), requirements 갱신, gitignore 갱신
- Tests 폴더 만들고 test 코드 작성

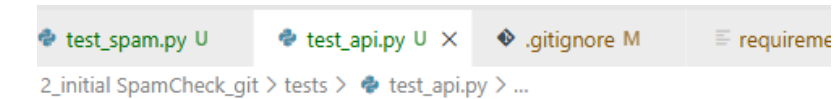


test_spam.py U × test_api.py U .gitignore M requirem

2_initial SpamCheck_git > tests > test_spam.py > ...

```
from app.spam import check_spam_rules
```

```
def test_check_spam_basic():  
    assert check_spam_rules("hello")[0] == "ham"  
    assert check_spam_rules("free prize click")[0] == "spam"
```



test_spam.py U test_api.py U × .gitignore M requirem

2_initial SpamCheck_git > tests > test_api.py > ...

```
from fastapi.testclient import TestClient  
from app.main import app
```

```
client = TestClient(app)
```

```
def test_classify_api_contract():  
    r = client.post("/classify", json={"text": "hello"})  
    assert r.status_code == 200  
    data = r.json()  
    assert "label" in data and "score" in data
```

테스트 자동화 필요성

• 테스트 코드 실행 (테스트 자동화) 해보기

• pytest

- os나 환경에 따라 우측 에러 생길 수 있음
- 특히 window + anaconda 조합

• Tests/conftest.py 작성

- Pytest가 실행 전 읽는 파일

• 다시 pytest

```
conftest.py U x test_spam.py U test_api.py U .gitignore M
2_initial SpamCheck_git > tests > conftest.py
# tests/conftest.py
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
```

```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_initial SpamCheck_git>pytest
===== test session starts =====
platform win32 -- Python 3.13.4, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_initial SpamCheck_git
plugins: anyio-4.12.1
collected 0 items / 2 errors

===== ERRORS =====
ERROR collecting tests/test_api.py
ImportError while importing test module 'C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_initial Spam
Hint: make sure your test modules/packages have valid Python names.
Traceback:
C:\Users\user\anaconda3\envs\class_py\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
tests\test_api.py:2: in <module>
    from app.main import app
E ModuleNotFoundError: No module named 'app'

ERROR collecting tests/test_spam.py
ImportError while importing test module 'C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_initial Spam
Hint: make sure your test modules/packages have valid Python names.
Traceback:
C:\Users\user\anaconda3\envs\class_py\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
tests\test_spam.py:1: in <module>
    from app.spam import check_spam
E ModuleNotFoundError: No module named 'app'

===== short test summary info =====
ERROR tests/test_api.py
ERROR tests/test_spam.py
!!!!!!!!!!!!!!!!!!!! Interrupted: 2 errors during collection !!!!!
===== 2 errors in 0.29s =====
```

```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_initial SpamCheck_git>pytest
===== test session starts =====
platform win32 -- Python 3.13.4, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_initial SpamCheck_git
plugins: anyio-4.12.1
collected 2 items

tests\test_api.py .
tests\test_spam.py .

===== 2 passed in 0.23s =====
```

테스트 자동화 필요성

- 코드에 버그가 있다면 테스트 결과는?

- 일부러 버그 심기 : 뭘 넣어도 spam
- 에러가 뜬다

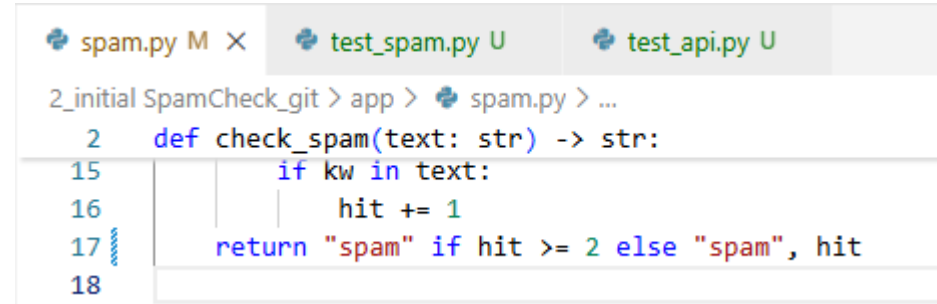
```
tests\test_api.py .
tests\test_spam.py F
```

```
===== FAILURES =====
test_check_spam_basic _

def test_check_spam_basic():
>     assert check_spam("hello")[0] == "ham"
E     AssertionError: assert 'spam' == 'ham'
E
E     - ham
E     + spam

tests\test_spam.py:4: AssertionError
===== short test summary info =====
FAILED tests/test_spam.py::test_check_spam_basic - AssertionError: assert 'spam' == 'ham'
===== 1 failed, 1 passed in 0.30s =====
```

- 개발자가 코드 변경하고 커밋 전 전체 테스트 돌려서 문제가 생기면 → 고치고 모두 pass할 때 까지 테스트 → 이걸 커밋하고 머지하자!
- 코드 원상 복구시키기



```
spam.py M x test_spam.py U test_api.py U
2_initial SpamCheck_git > app > spam.py > ...
2 def check_spam(text: str) -> str:
15     if kw in text:
16         hit += 1
17     return "spam" if hit >= 2 else "spam", hit
18
```

테스트 자동화 필요성

- 코드 원상 복구하고 다시 커밋-머지

- git add .
- git commit -m "add tests"
- git push origin main

```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_initial SpamCheck_git>git add .
```

```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_initial SpamCheck_git>git commit -m "add test"
[main 2ce75dc] add test
5 files changed, 24 insertions(+), 1 deletion(-)
create mode 100644 tests/conftest.py
create mode 100644 tests/test_api.py
create mode 100644 tests/test_spam.py
```

```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_initial SpamCheck_git>git push origin main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 32 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (8/8), 1015 bytes | 1015.00 KiB/s, done.
Total 8 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/MisookKim/jnu2603-spamchecker_test.git
e4e71de..2ce75dc main -> main
```

```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_initial SpamCheck_git>
```

```
spam.py x test_spam.py U test_api.py U
2_initial SpamCheck_git > app > spam.py > ...
2 def check_spam(text: str) -> str:
13     hit = 0
14     for kw in spam_keywords:
15         if kw in text:
16             hit += 1
17     return "spam" if hit >= 2 else "ham", hit
18
```

jnu2603-spamchecker_test Private Watch 0

main Go to file + Code

MisookKim	add test	2ce75dc · now	14 Commits
app	add file name	26 minutes ago	
static	add file name	26 minutes ago	
tests	add test	now	
.gitignore	add test	now	
requirements.txt	add test	now	

README

테스트 자동화

- 개발자가 수동으로 체크할 필요 없음!
- 기능 추가 → 테스트 코드 작성/실행 → 원하는 대로 고쳐졌는지 + 기존 기능 제대로는지 자동으로 확인
→ 문제 있으면 고치고 다시 테스트 → 테스트 pass 시 커밋 후 머지
- 문제: 개발자 개인 PC 기준의 자동화!, 가끔 바빠서 테스트 안 돌리고 push 할 수 있음
- ▶ Git → GitHub Main에 합쳐질 때 마다, 테스트를 자동으로 해야함!
 - = push/PR 때 마다 자동으로 테스트 되도록 해야함
 - = Continuous Integration: 코드가 통합되는 순간마다 자동으로 빌드/테스트를 해서
깨진 코드가 main에 들어오도록 막기 = 지속적으로 안전하게 통합시키자!

GitHub Action 기반 CI

- GitHub 안에서 “특정 이벤트가 발생하면 자동으로 명령을 실행해주는 시스템”

- GitHub가 자동으로 컴퓨터(가상머신)을 빌려주고
- 우리가 정해둔 순서대로 (스크립트/코드 작성)
 - 파이썬 설치
 - 라이브러리 설치
 - 테스트 실행
 - 과 같은 작업을 사람 대신 해 줌

= 개발자가 머지 전 테스트를 까먹어도 머지시킬려고 할 때 알아서 해줄 수 있다!

GitHub Action 기반 CI

- 기본 구조

- 이벤트 → 워크플로우 → 잡 → 스텝
 - 이벤트: push, pull_request
 - 워크플로우: CI
 - 잡: 테스트 실행
 - 스텝: 코드 받기 → 파이썬 설치 → pytest 실행

- GitHub Actions는 CI 서버

GitHub Action 기반 CI

- Action 파일 만들기

- .github/workflows/ci.yml

- .github은 깃허브가 이벤트를 모니터링하며
자동으로 실행시키기 위한 파일들의 모음

- 우측 내용으로 작성 후 git에 커밋 후 push

- git add .
 - git commit -m “Use GitHub Actions for CI”

```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(:
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 32 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 609 bytes | 609.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/MisookKim/jnu2603-spamchecker_test.git
2ce75dc..334b279  main -> main
```

```
name: CI
```

```
on:
```

```
  push:
```

```
    branches: [ "main" ]
```

```
  pull_request:
```

```
    branches: [ "main" ]
```

```
jobs:
```

```
  test:
```

```
    runs-on: ubuntu-latest
```

```
  steps:
```

```
    - name: Checkout code
```

```
      uses: actions/checkout@v4
```

```
    - name: Set up Python
```

```
      uses: actions/setup-python@v5
```

```
      with:
```

```
        python-version: "3.10"
```

```
    - name: Install dependencies
```

```
      run: |
```

```
        pip install --upgrade pip
```

```
        pip install -r requirements.txt
```

```
    - name: Run tests
```

```
      run: pytest
```

GitHub Action 기반 CI

- GitHub Action 둘러보기

- Green check 생김

The collage consists of four screenshots illustrating the GitHub Actions workflow process:

- Top Left:** The repository page for `misookim / jnu2603-spamchecker_test`. The `Actions` tab is selected, showing a workflow named `Use GitHub Actions for CI` with a green checkmark and commit `334b279`.
- Top Right:** A commit view for `334b279` showing the workflow `Use GitHub Actions for CI` on the `main` branch. It highlights a file change in `.github/workflows/ci.yml`.
- Bottom Left:** The `Actions` page showing a list of workflows. The `CI` workflow is selected, displaying a table of workflow runs. The most recent run, `Use GitHub Actions for CI #1`, is shown as successful (green checkmark) and completed 1 minute ago.
- Bottom Right:** A detailed view of the workflow run `Use GitHub Actions for CI #1`. It shows the summary, jobs (including a `test` job), and the workflow file `ci.yml` which is triggered on push. The `test` job is shown as successful in 15s.

GitHub Action 기반 CI

• GitHub Action 둘러보기

Summary

All jobs

test

Run details

Usage

Workflow file

test

succeeded 2 minutes ago in 15s

Set up job

Checkout code

Set up Python

Install dependencies

Run tests

Post Set up Python

Post Checkout code

Complete job

Set up job

1 Current runner version: '2.331.0'

2 ▶ Runner Image Provisioner

9 ▶ Operating System

13 ▶ Runner Image

18 ▶ GITHUB_TOKEN Permissions

22 Secret source: Actions

23 Prepare workflow directory

24 Prepare all required actions

25 Getting action download info

26 Download action repository 'actions/checkout@v4'

27 Download action repository 'actions/setup-python@v5'

28 Complete job name: test

Checkout code

1 ▶ Run actions/checkout@v4

16 Syncing repository: MisookKim/jnu2603-spamchecker_test

17 ▶ Getting Git version info

21 Temporarily overriding HOME='/home/runner/work/_temp/00a0509cea2290a0' before making global git config changes

22 Adding repository directory to the temporary git global config

23 /usr/bin/git config --global --add safe.directory /home/runner/work/jnu2603-spamchecker_test/jnu2603-spamchecker_test

24 Deleting the contents of '/home/runner/work/jnu2603-spamchecker_test'

25 ▶ Initializing the repository

42 ▶ Disabling automatic garbage collection

44 ▶ Setting up auth

52 ▶ Fetching the repository

56 ▶ Determining the checkout info

57 /usr/bin/git sparse-checkout disable

58 /usr/bin/git config --local --unset-all extensions.worktree

59 ▶ Checking out the ref

63 /usr/bin/git log -1 --format=%H

64 334b2795e288d5dc50c6e6d3a13e62567227ead

Set up Python

1 ▶ Run actions/setup-python@v5

9 ▼ Installed versions

10 Successfully set up CPython (3.10.19)

Install dependencies

1 ▶ Run pip install --upgrade pip

12 Requirement already satisfied: pip in

13 /opt/hostedtoolcache/Python/3.10.19/x64/lib/python3.10

14 Collecting pip

15 Downloading pip-26.0.1-py3-none-any.whl (1.8 MB)

16 ===== 1.8/1.8 MB

17 Installing collected packages: pip

18 Attempting uninstall: pip

19 Found existing installation: pip 26.0

20 Uninstalling pip-26.0:

21 Successfully uninstalled pip-26.0

22 Successfully installed pip-26.0.1

23 Collecting fastapi (from -r requirements.txt (line 1))

24 Downloading fastapi-0.128.7-py3-none-any.whl.metadata

25 Collecting uvicorn (from -r requirements.txt (line 2))

26 Downloading uvicorn-0.40.0-py3-none-any.whl.metadata

27 Collecting python-multipart (from -r requirements.txt

28 Downloading python_multipart-0.0.22-py3-none-any.whl

29 Collecting pydantic (from -r requirements.txt (line 4))

30 Downloading pydantic-2.12.5-py3-none-any.whl.metadata

31 Collecting pytest (from -r requirements.txt (line 5))

32 Downloading pytest-9.0.2-py3-none-any.whl.metadata

33 Collecting httpx (from -r requirements.txt (line 6))

34 Downloading httpx-0.28.1-py3-none-any.whl.metadata

35 Collecting starlette<1.0.0,>=0.40.0 (from fastapi->-r

36 Downloading starlette-0.52.1-py3-none-any.whl.metac

37 Collecting typing-extensions>=4.8.0 (from fastapi->-r

38 Downloading typing_extensions-4.15.0-py3-none-any.w

39 Collecting typing-inspection>=0.4.2 (from fastapi->-r

40 Downloading typing_inspection-0.4.2-py3-none-any.wh

41 Collecting annotated-doc>=0.0.2 (from fastapi->-r req

Run tests

1 ▶ Run pytest

11 ===== test session start

12 platform linux -- Python 3.10.19, pytest-9.0.2,

13 rootdir: /home/runner/work/jnu2603-spamchecker_t

14 plugins: anyio-4.12.1

15 collected 2 items

16

17 tests/test_api.py .

18 tests/test_spam.py .

19

20 ===== 2 passed in 0.39s

Post Set up Python

1 Post job cleanup.

Post Checkout code

1 Post job cleanup.

2 /usr/bin/git version

3 git version 2.52.0

4 Temporarily overriding HOME='/home/runner/work/_temp/00a0509cea2290a0' before making global git config changes

5 Adding repository directory to the temporary git global config

6 /usr/bin/git config --global --add safe.directory /home/runner/work/jnu2603-spamchecker_test/jnu2603-spamchecker_test

7 /usr/bin/git config --local --name-only --get-regexp 'core.sshCommand' && git config --local --name-only --get-regexp 'http\https\:\.\github\com\.\extraheader

8 /usr/bin/git submodule foreach --recursive sh -c 'http\https\:\.\github\com\.\extraheader' || :

9 /usr/bin/git config --local --name-only --get-regexp 'http\https\:\.\github\com\.\extraheader' || :

10 http\https\:\.\github\com\.\extraheader

11 /usr/bin/git config --local --unset-all http\https\:\.\github\com\.\extraheader' || :

12 /usr/bin/git submodule foreach --recursive sh -c 'http\https\:\.\github\com\.\extraheader' || :

13 /usr/bin/git config --local --name-only --get-regexp 'http\https\:\.\github\com\.\extraheader' || :

14 /usr/bin/git submodule foreach --recursive git config --get-regexp remote.origin.url

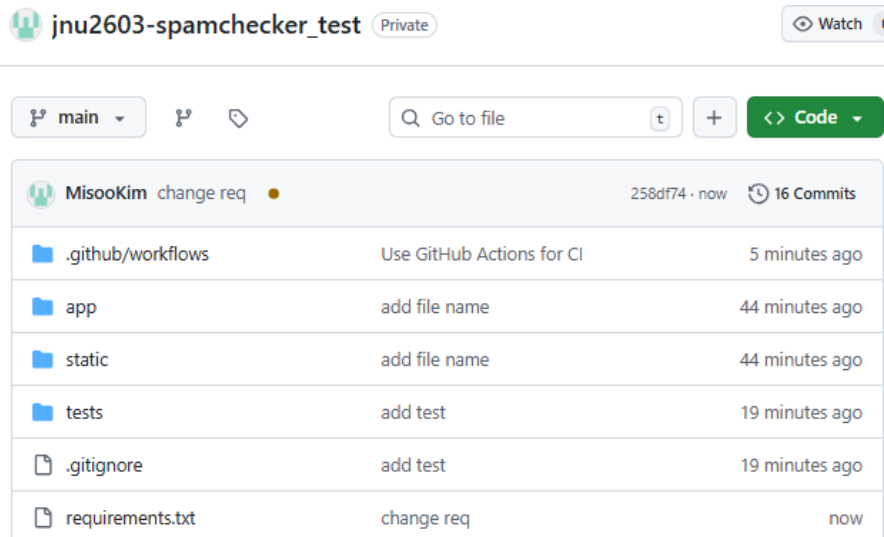
Complete job

1 Cleaning up orphan processes

 **신남대학교**
CHONNAM NATIONAL UNIVERSITY

GitHub Action 기반 CI

• 회귀 유도해서 CI 결과 확인해보기



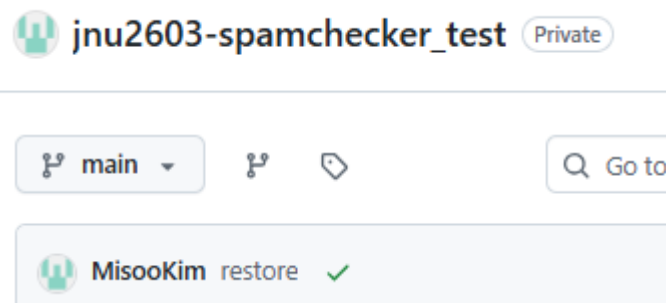
Repository: jnu2603-spamchecker_test (Private)

Commit: MisooKim change req (258df74 · now) 16 Commits

File	Change	Time
.github/workflows	Use GitHub Actions for CI	5 minutes ago
app	add file name	44 minutes ago
static	add file name	44 minutes ago
tests	add test	19 minutes ago
.gitignore	add test	19 minutes ago
requirements.txt	change req	now

• 다시 복구 후 커밋

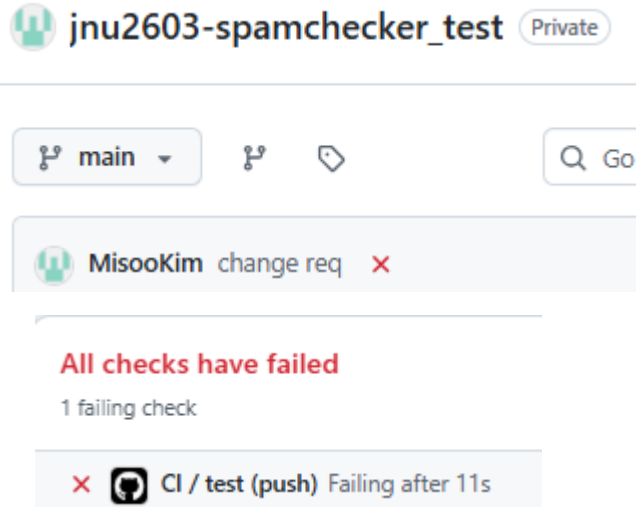
```
requirements.txt
2_initial SpamCheck_git > requireme
1 fastapi
2 uvicorn
3 python-multipart
4 pydantic
5 pytest
6 httpx
```



Repository: jnu2603-spamchecker_test (Private)

Commit: MisooKim restore ✓

```
requirements.txt
2_initial SpamCheck_git > requireme
1 fastapi
2 uvicorn
3 python-multipart
```

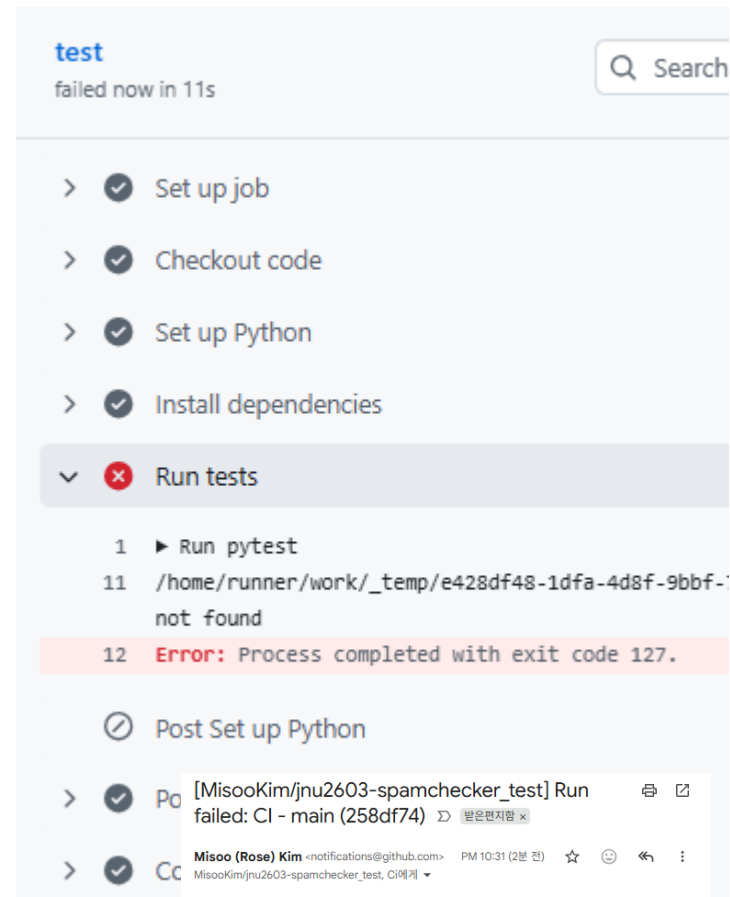


Repository: jnu2603-spamchecker_test (Private)

Commit: MisooKim change req ✗

All checks have failed
1 failing check

✗ CI / test (push) Failing after 11s



Workflow: test (failed now in 11s)

- Set up job
- Checkout code
- Set up Python
- Install dependencies
- Run tests (Failed)

1 Run pytest
11 /home/runner/work/_temp/e428df48-1dfa-4d8f-9bbf-... not found
12 Error: Process completed with exit code 127.

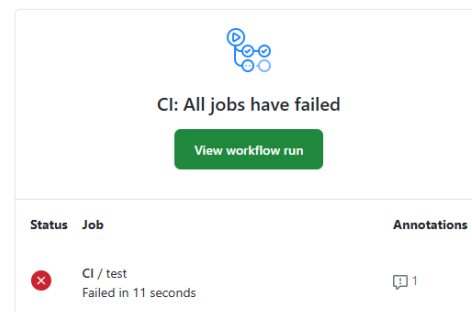
Post Set up Python

[MisooKim/jnu2603-spamchecker_test] Run failed: CI - main (258df74) ✗

Misoo (Rose) Kim <notifications@github.com> PM 10:31 (2분 전) ☆ ☺ ⏪ ⋮



[MisooKim/jnu2603-spamchecker_test] CI workflow run



CI: All jobs have failed

[View workflow run](#)

Status	Job	Annotations
✗	CI / test Failed in 11 seconds	1

GitHub Action 기반 CI

- 아까 그 문제 PR로 해결해보자!

- 문제 원인

- 프론트 입장에서 json 보냄 → 서버입장에서 정확히 뭔지 모름
- Pydantic 규칙 상 text 필드를 반드시 가져야함을 명시
- FastAPI(서버)는 들어온게 뭔지 정확히 몰라서 JSON으로 파싱 안함
 - 전에는 dict라고 명시했음. Pydantic 도입으로 문서 개선했으나 에러 발생

```
19 class ClassifyRequest(BaseModel):
20     text: str
21
22 @app.post("/classify")
23 async def classify(payload: ClassifyRequest):
24     text = payload.text
25     label, score = check_spam(text)
26     return {
27         "label": label, "score": score
28     }
```

```
main.py M x  <> index.html M ●
2_initial SpamCheck_git > static > <> index.html > html > body > script > classify
3  <html>
8  <body>
17  <script>
18      function classify() {
26          // 브라우저가 서버에 HTTP 요청을 보내는 함수
27          fetch("/classify", {
28              method: "POST",
29              headers: {"Content-Type": "application/json"}, // fix
30              body: JSON.stringify({text})
31          })
```

스팸 판별기

스팸으로 의심되는 메시지를 붙여넣어주세요!

스팸 탐지

메시지를 입력해 주세요!

스팸 판별기

ddd

스팸 탐지

결과: undefined (score=undefined)

```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_
INFO: Will watch for changes in these directories: ['C:\Users\user\OneDrive\
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [3520] using StatReload
INFO: Started server process [2424]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:64831 - "GET / HTTP/1.1" 200 OK
INFO: 127.0.0.1:51491 - "POST /classify HTTP/1.1" 422 Unprocessable Content
```

```
...
async def classify(
    payload: dict = Body(..., example={"te
}):
```

GitHub Action 기반 CI

- 아까 그 문제 PR로 해결해보자!

- git switch -c fix/all-undefined
- (코드 수정)
- git add .
- git commit -m “fix 422 by sending JSON content-type header”
- git push -u origin fix/all-undefined

```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_initial Spam
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 32 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 450 bytes | 450.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote:
remote: Create a pull request for 'fix/all-undefined' on GitHub by visiting:
remote:   https://github.com/MisookKim/jnu2603-spamchecker_test/pull/new/fix/all-undefined
remote:
To https://github.com/MisookKim/jnu2603-spamchecker_test.git
 * [new branch]      fix/all-undefined -> fix/all-undefined
branch 'fix/all-undefined' set up to track 'origin/fix/all-undefined'.
```

```
main.py M x  <> index.html M •
2_initial SpamCheck_git > static > <> index.html > html > body > script > classify
3  <html>
8  <body>
17  <script>
18  function classify() {
26      // 브라우저가 서버에 HTTP 요청을 보내는 함수
27      fetch("/classify", {
28          method: "POST",
29          headers: { "Content-Type": "application/json" }, // fix
30          body: JSON.stringify({text})
31      })
```

스팸 판별기

미건 스팸?

스팸 탐지

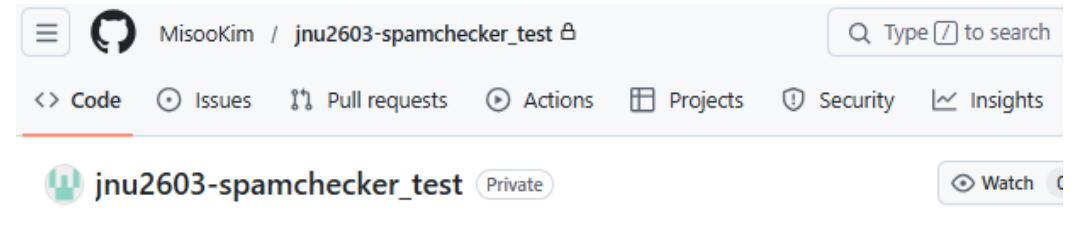
결과: ham (score=0)

스팸 판별기

Hello Prize Free

스팸 탐지

결과: spam (score=2)



GitHub Action 기반 CI

• 아까 그 문제 PR로 해결해보자!

fix 422 by sending JSON conte

MisooKim wants to merge 1 commit into `main` from `f`

Conversation 0 Commits 1 Checks 0

MisooKim commented now

What: Add Content-Type: application/json header to classi

Why: FastAPI Pydantic body binding requires JSON conten

Test: pytest locally + CI should pass

`fix 422 by sending JSON content-type header`

Some checks haven't completed yet
1 in progress check

CI / test (pull_request) Started now — This check has

No conflicts with base branch
Merging can be performed automatically.

Merge pull request You can also merge this with the [View command line instructions.](#)

fix 422 by sending JSON content-type header #5

MisooKim wants to merge 1 commit into `main` from `fix/all-undefined`

Conversation 0 Commits 1 Checks 0 Files changed 1

MisooKim commented now

What: Add Content-Type: application/json header to classify request

Why: FastAPI Pydantic body binding requires JSON content-type

Test: pytest locally + CI should pass

`fix 422 by sending JSON content-type header` `07c85da`

All checks have passed
1 successful check

CI / test (pull_request) Successful in 16s

No conflicts with base branch
Merging can be performed automatically.

Merge pull request You can also merge this with the command line. [View command line instructions.](#)

[← Back to pull request #5](#)

fix 422 by sending JSON content-type header #4

Re-run all jobs

Summary

All jobs

test

Run details

Usage

Workflow file

test

succeeded now in 16s

Search logs

```
>  Set up job 1
>  Checkout code 1
>  Set up Python 0
>  Install dependencies 9
>  Run tests 1
  1 ▶ Run pytest
 11 ===== test session starts =====
 12 platform linux -- Python 3.10.19, pytest-9.0.2, pluggy-1.6.0
 13 rootdir: /home/runner/work/jnu2603-spamchecker_test/jnu2603-spamchecker_test
 14 plugins: anyio-4.12.1
 15 collected 2 items
 16
 17 tests/test_api.py . [ 50%]
 18 tests/test_spam.py . [100%]
 19
 20 ===== 2 passed in 0.54s =====
```

MisooKim merged commit `1eb1f24` into `main` now
1 check passed

View details

Revert



Pull request successfully merged and closed

You're all set — the `fix/all-undefined` branch can be safely deleted.

Delete branch



Pull request successfully merged and closed

You're all set — the branch has been merged.

GitHub Action 기반 CI

- 원격 브랜치 지웠으니 로컬도 잊지말고

- git branch
- git switch main
- git pull (받아오기)
- git branch
- git branch -D fix/all-undefined
- git fetch -p

```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_initial SpamCheck_git>git branch
* fix/all-undefined
  main
```

```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_initial SpamCheck_git>git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
```

```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_initial SpamCheck_git>git pull
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (1/1), 930 bytes | 310.00 KiB/s, done.
From https://github.com/MisookKim/jnu2603-spamchecker_test
   c324934..1eb1f24  main       -> origin/main
Updating c324934..1eb1f24
Fast-forward
 static/index.html | 1 +
 1 file changed, 1 insertion(+)
```

```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_initial SpamCheck_git>git branch
fix/all-undefined
* main
```

```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_initial SpamCheck_git>git branch -D fix/all-undefined
```

```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_initial SpamCheck_git>git fetch -p
From https://github.com/MisookKim/jnu2603-spamchecker_test
 - [deleted]          (none)      -> origin/fix/all-undefined
```

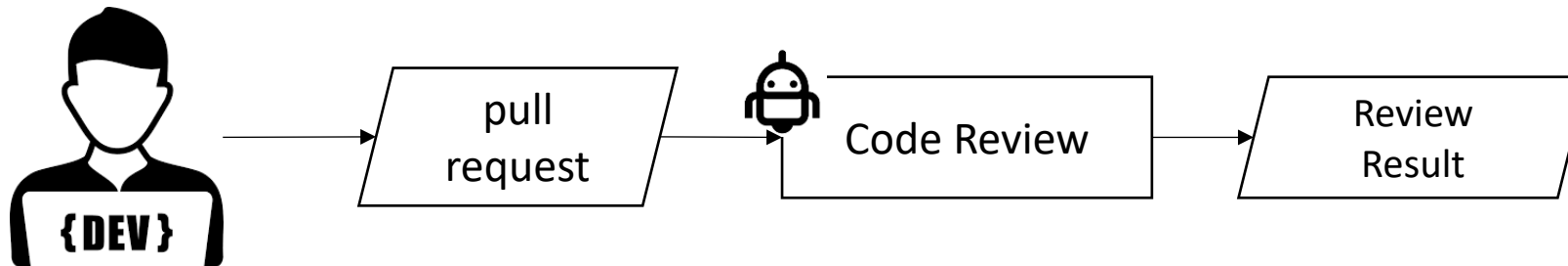
- = git switch main && git pull && git branch -d <브랜치> && git fetch -p

그 외에도 GitHub Actions를 통해 할 수 있는 것

- 자동 이슈 코멘터 봇



- 코드 리뷰



그 외에도 GitHub Actions를 통해 할 수 있는 것

• 자동 이슈 코멘터 봇

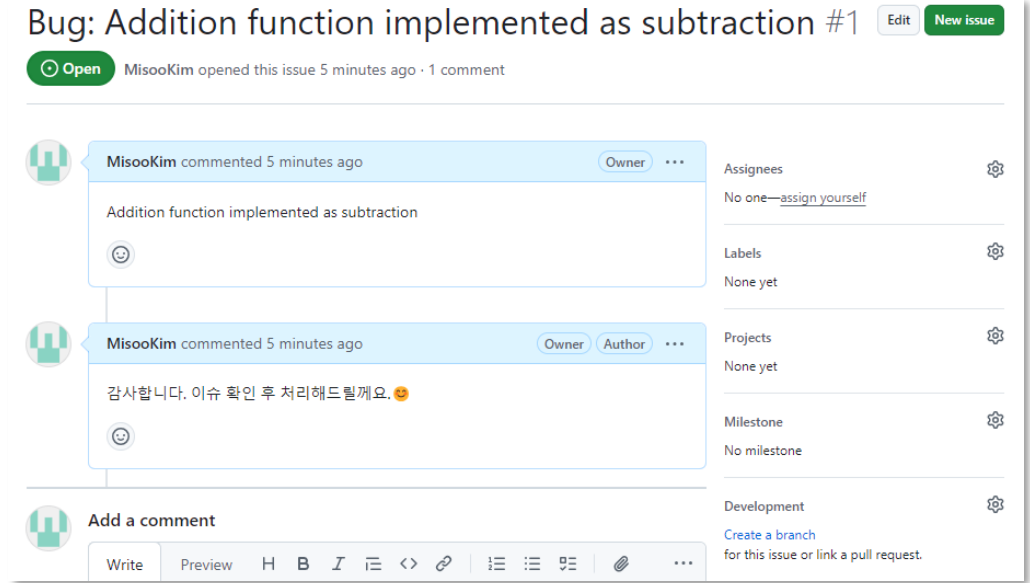
• issue_commenter.yml : 자동 답변

```
name: Auto Comment on Issue

# 이슈가 생성될 때 이벤트 트리거
on:
  issues:
    types:
      - opened # 이슈가 생성될 때만 반응

jobs:
  comment:
    runs-on: ubuntu-latest

    steps:
      # GitHub 토큰을 사용하여 이슈에 코멘트 달기
      - name: Comment on new issue
        uses: actions/github-script@v6
        with:
          github-token: ${{ secrets.MY_PERSONAL_TOKEN }} # GitHub에서 제공하는 기본 토큰 사용
          script: |
            github.rest.issues.createComment({
              issue_number: context.payload.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: "감사합니다. 이슈 확인 후 처리해드릴게요. 😊"
            })
```



그 외에도 GitHub Actions를 통해 할 수 있는 것

- 코드 리뷰

- <https://github.com/marketplace/actions/chatgpt-codereviewer>

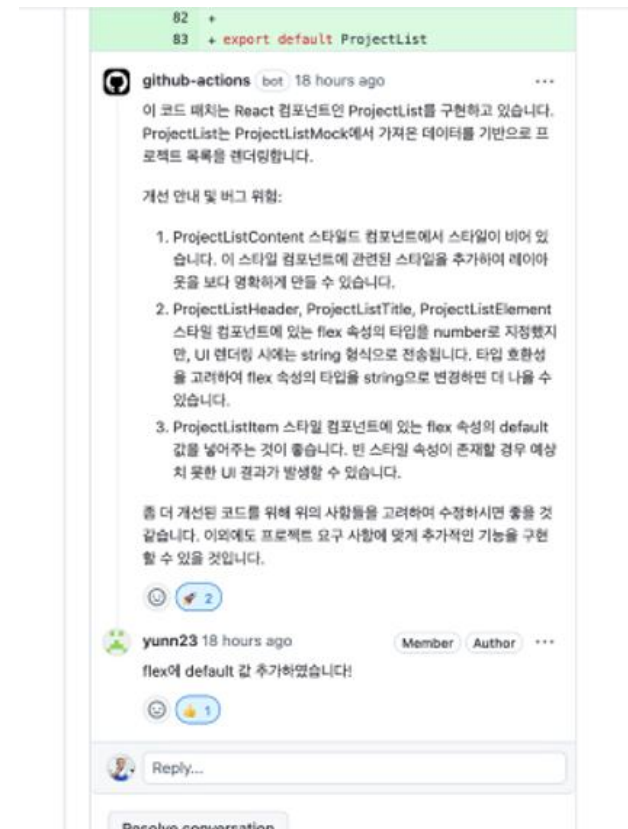
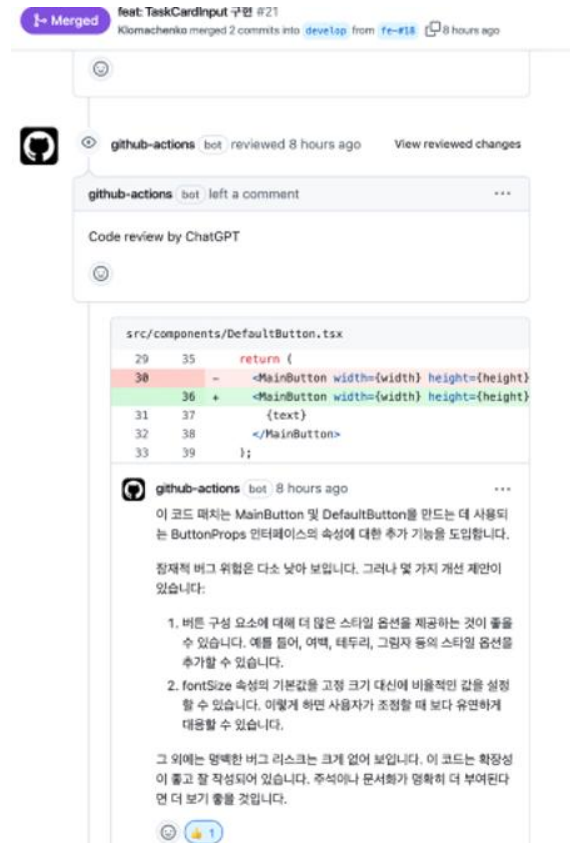
```
name: Code Review From ChatGPT
```

```
permissions:
  contents: read
  pull-requests: write
```

```
on:
  pull_request:
    types: [opened, reopened, synchronize]
```

```
jobs:
  code-review:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v3

      - name: Run ChatGPT Code Review
        uses: anc95/ChatGPT-CodeReview@main
        env:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
          OPENAI_API_KEY: ${ secrets.OPENAI_API_KEY }
          LANGUAGE: Korean
          MODEL: gpt-3.5-turbo
          ACTIONS_RUNNER_DEBUG: 'true'
```



그 외에도 GitHub Actions를 통해 할 수 있는 것

- PR에 댓글로 코드 커버리지 정보 요약해주기
- 정적 분석 붙여서 DevSecOps로 확장

codecov-commenter commented last month • edited

Codecov Report

Attention: 83 lines in your changes are missing coverage. Please review.

Comparison is base (c8700f4) 80.14% compared to head (073afe6) 75.41%. Report is 26 commits behind head on master.

Files	Patch %	Lines
keras/backend/tensorflow/linalg.py	60.43%	28 Missing and 8 partials
keras/ops/linalg.py	91.04%	18 Missing and 6 partials
keras/backend/numpy/linalg.py	86.04%	3 Missing and 3 partials
keras/backend/torch/linalg.py	84.21%	3 Missing and 3 partials
keras/backend/jax/linalg.py	90.47%	2 Missing and 2 partials
keras/callbacks/tensorboard.py	85.18%	1 Missing and 3 partials
keras/backend/jax/trainer.py	95.34%	0 Missing and 2 partials
keras/metrics/metric.py	80.00%	1 Missing

► Additional details and impacted files

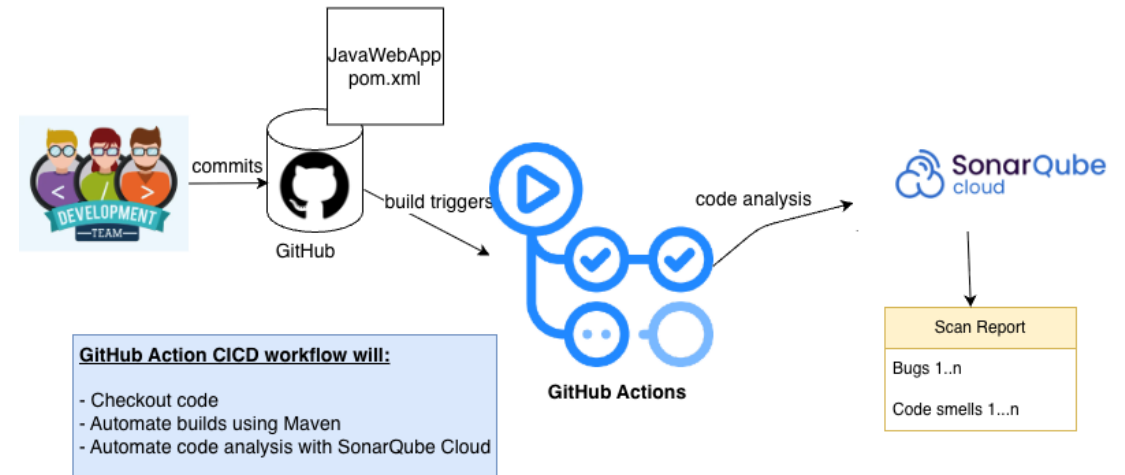
[View full report in Codecov by Sentry.](#)

[Have feedback on the report? Share it here.](#)

hertschuh requested a review from haifeng-jin last month

google-ml-butler bot added the awaiting review label last month

Automate Static Code Quality Analysis using GitHub Actions with SonarQube Cloud



숙제 5. 실습 9

- 이번 주 일요일 자정까지
- 자유 양식

1. 실습 9

- 목적, 방법, 결과 스크린샷 및 설명
- 나중에 GitHub와 연동하여 이력 확인할 것이니 매주 복습 잘하기